

Appunti di Electronics

LEZIONI DEL PROF. FANUCCI AA 2023-2024

SAVERIO MOSTI

Sommario

Recap di elettronica digitale	5
Cross-section of NMOS.....	6
NMOS as a switch	Errore. Il segnalibro non è definito.
PMOS as a switch.....	Errore. Il segnalibro non è definito.
Controlled switch.....	Errore. Il segnalibro non è definito.
Pass gate	7
Inverter	9
Propagation delay.....	Errore. Il segnalibro non è definito.
Processo di fabbricazione dei CMOS	11
Photolithography.....	11
Deposition and etching.....	Errore. Il segnalibro non è definito.
Inverter cross-section	Errore. Il segnalibro non è definito.
Fabrication steps.....	12
Design Rules.....	Errore. Il segnalibro non è definito.
Why Have Design Rules?	18
Electronic abstraction levels	18
Integrated circuit design style.....	18
Full-Custom Design	19
Layout	20
Design Rule Check [DRC]	20
Regole di progettazione	20
CAD	20
Layout vs Schematic [LVS].....	21
Layout Parasitic Extraction [LPE].....	21
Semi-Custom Design.....	22
Gate Array Structure.....	22
Matrici di porte logiche	22
Personalizzazione limitata	22
Vantaggi della Gate Array Structure	22
Standard Cell Structure.....	23
Librerie di celle standard	23
Personalizzazione delle interconnessioni	23
Vantaggi della Standard Cell Structure	23
Svantaggi della Standard Cell Structure.....	23
ASIC (Application-Specific Integrated Circuit).....	24

Single gate Structure.....	24
Gate array interconnection.....	24
Matrice di porte Logiche programmabili.....	24
Interconnessione programmabile.....	24
Efficienza e Flessibilità	24
Sea-of-gate structure	25
Matrice Regolare	25
Interconnessione Programmabile.....	25
Efficienza	25
Applicazioni.....	25
Register Transfer Level [RTL].....	26
Functional Verification	26
Logic Synthesis.....	27
Place and Route	27
Costo di un circuito integrato	28
Non-Recurrent engineering Cost [NRE]	28
Recurrent Engineering Cost [RE].....	28
Costo totale di produzione	28
Costo per unità	28
Come risparmiare	28
FPGA (Field Programmable Gate-Array unit).....	29
Da cosa sono fatti?.....	29
Vantaggi di FPGA.....	29
Programmare un FPGA	29
FPGA SRAM Base (Xilinx)	29
FPGA basato su tecnologie anti-fused	31
Riassunto degli stili di design dei circuiti integrati.....	32
The “Energy-Flexibility” Gap.....	32
SoC (System-on-Chip)	33
La foundry library.....	33
Tipologie di Logica	34
Logica combinatoria	34
Logica sequenziale.....	34
Macchina di Mealy	34
Macchina di Moore.....	35
Nodi	36

Hard Node.....	36
Soft Node	36
Databook di un nodo	36
Propagation delay	36
Alimentazione e ritardo di propagazione	37
Soglia e ritardo di propagazione	37
Interconnessione	38
Strenght	38
Leakage	39
Layout e cell area.....	39
Wafer Cross-Section.....	39
Symbol of the node	39
True Table	39
Capacità	39
Strength	39
Transition Time (Ritardo di propagazione)	40
Technology corners.....	40
Typical	40
Worst	40
Best	40
Rumore	41
Comportamento statico (Steady state parameters)	41
Soglie	41
Margini di rumore.....	42
Transistors.....	42
CMOS Complementare	42
Pull-down Network (PDN)	42
Pull-up Network (PUN)	43
NAND2	43
NOR2.....	44
Funzioni complesse (Teoria della dualità).....	44
XOR	45
Complementary CMOS e il ritardo di propagazione	45
Analisi Dinamica (NAND)	47
Percorso critici di una rete combinatoria	50
CMOS gate – Body effect	52

Fan-in	54
Esempio di analisi dinamica (NAND)	55
Catena di inverter	56
Timing	57
Latch	57
Flip Flop Edge Triggered (Registro)	57
Clock Skew	57
Gals (Global Asynchronous Locally Synchronous)	57
Timing classification.....	58
Tsetup e Thold	58
Tc-q	58
Tplogic.....	58
Timing violations.....	59
Setup time violation.....	59
Hold time violation	60
Come evitarle?.....	60
Clock Queue.....	60
Timing e strumenti di sintetizzazione	61
Sources of clock Skew and Jitter in a clock network.....	62
Clock skew	63
Clock Jitter	64
Clock skew e jitter combinati	65
Generatore di Clock	66
Nota sulla Zibo board.....	66
PPL (Anello di Blocco di Fase)	66
Distribuire il clock su una rete	67

Introduzione

Questi appunti sono nati da una grande disperazione, acidi e voglia di picchiare. L'uso di intelligenza artificiale è stato massivo ma mantenendo sempre una certa eleganza.

Veramente, col cuore, vi auguro uno studio decente.

I primi capitoli non servono ad una mazza...

Recap di elettronica digitale

Cross-section di un NMOS

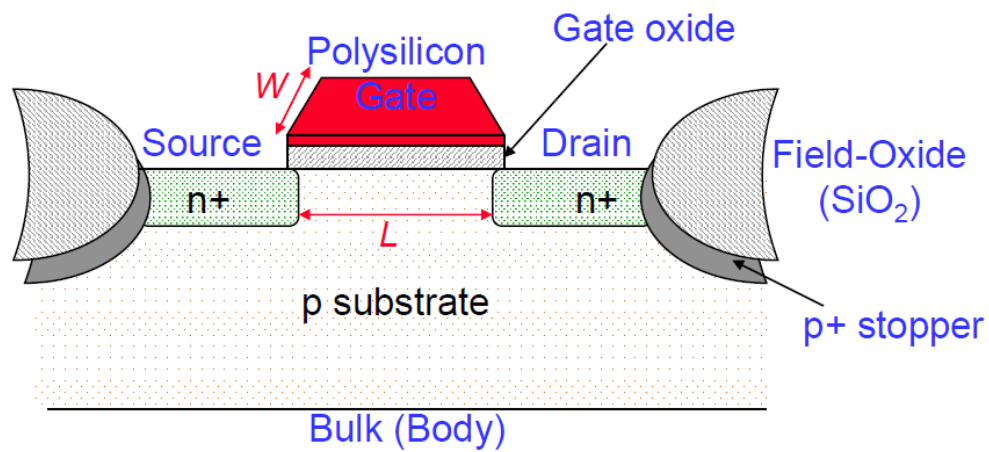


Figure 1- Struttura di un transistor.

Approssimazione di un NMOS ad uno switch

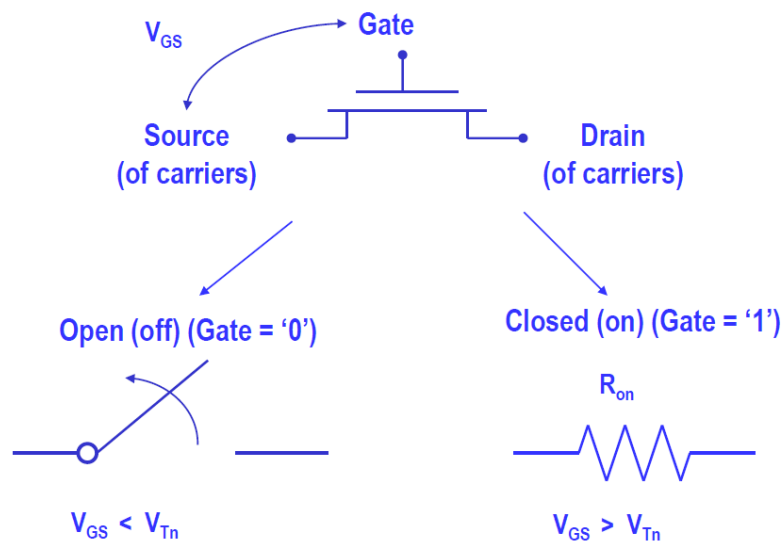


Figure 2- Approssimazione NMOS

Approssimazione di un PMOS ad uno switch

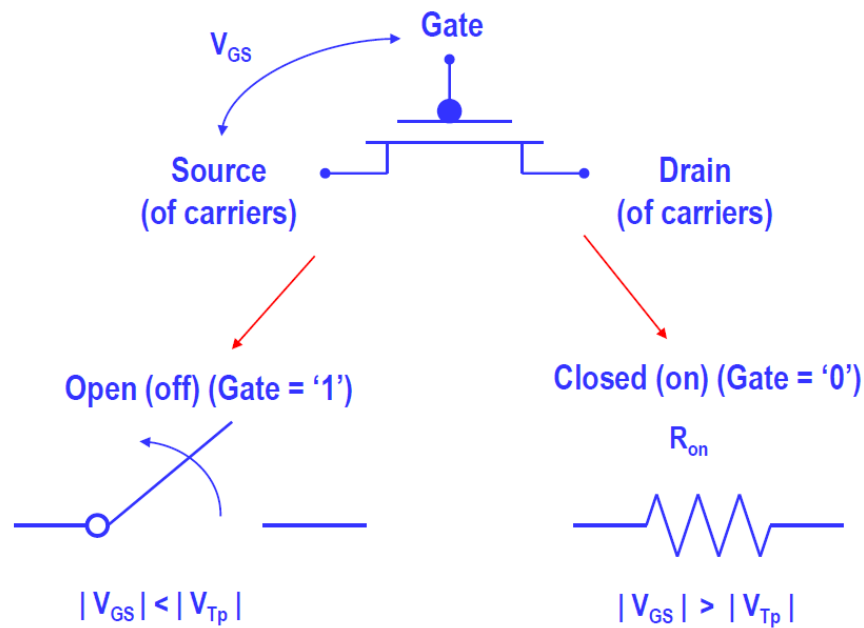
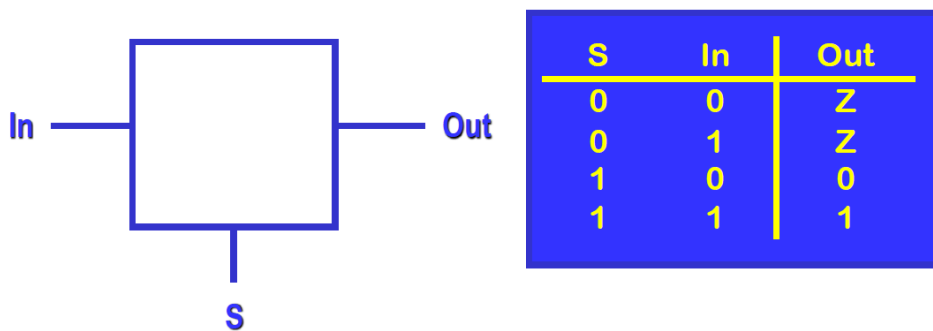


Figure 3 - Approssimazione PMOS

Pass gate

Three-state logic gates: high (H), low (L) and high-impedance (Z)



La resistenza equivalente di questo dispositivo è la seguente:

$$I_D = \beta_n \left[(V_{GS} - V_T) V_{DS} - \frac{V_{DS}^2}{2} \right] \approx \beta_n (V_{GS} - V_T) V_{DS}$$

$$\frac{1}{R} = \frac{dI_D}{dV_{DS}} = \beta_n (V_{GS} - V_T)$$

$$R = \frac{1}{\beta_n (V_{GS} - V_T)} = \frac{1}{\beta_n (4 - V_U)}$$

$V_T = 1V$

Figure 4 - Calcolo resistenza equivalente

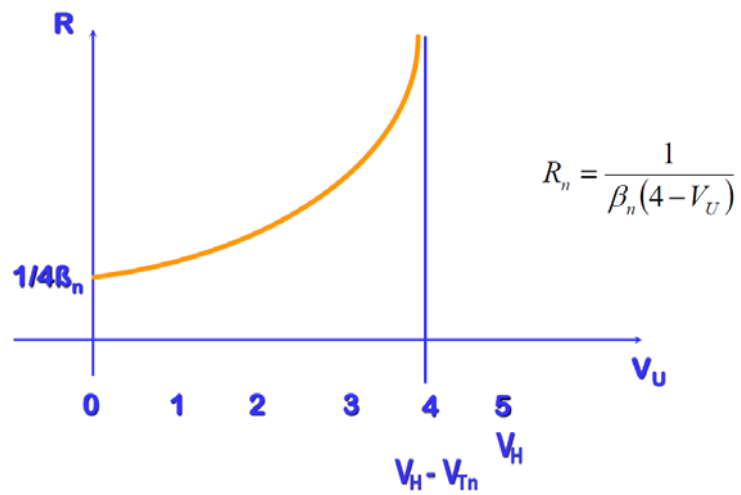
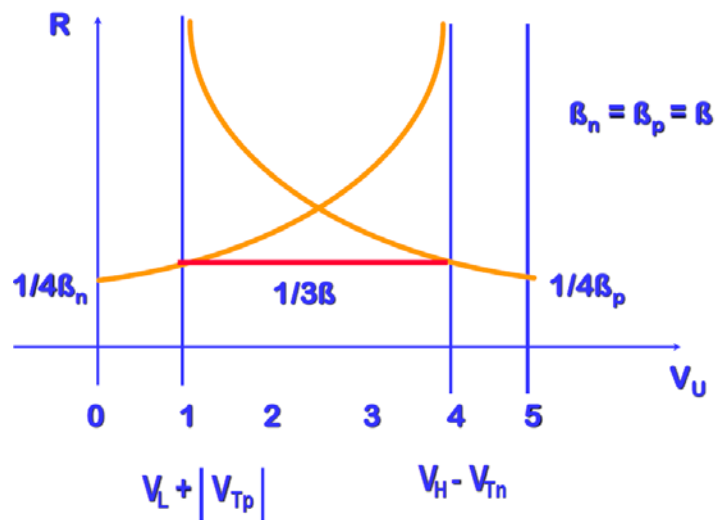
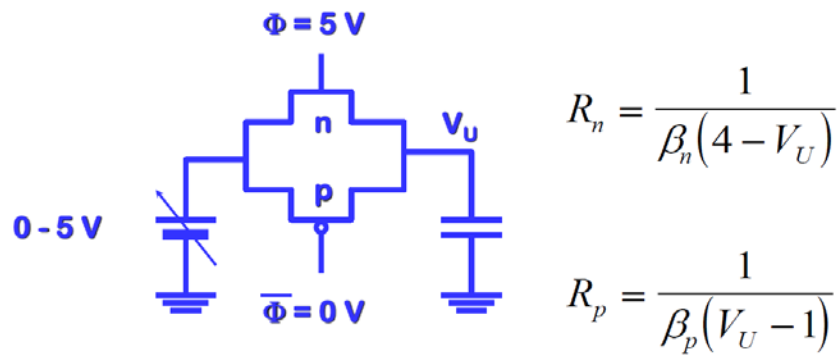


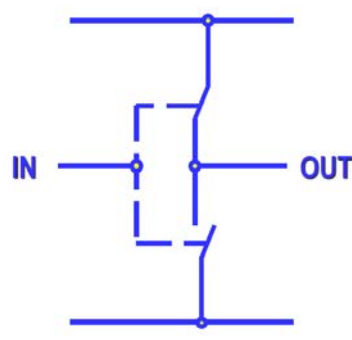
Figure 5 - Come varia la resistenza equivalente.

La resistenza equivalente varia in funzione della tensione di ingresso.



Inverter realizzato con Switch e CMOS

Switch based idea



CMOS

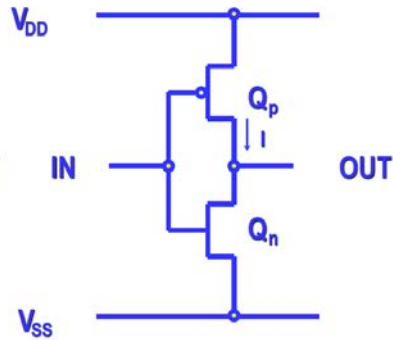
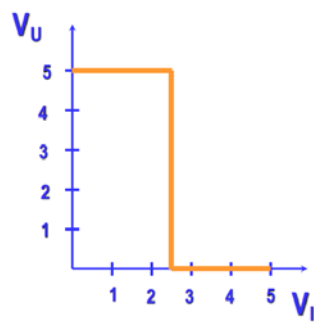


Figure 6 - Le due implementazioni degli inverter.

Il comportamento dell'uscita dipende (ovviamente) dalla tensione di ingresso.

IDEAL



REAL

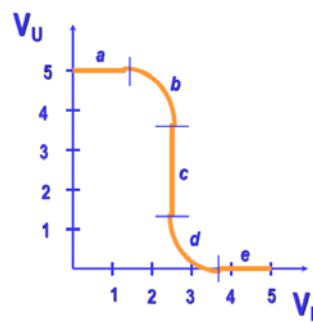


Figure 7- Caratteristica ideale e reale dell'inverter.

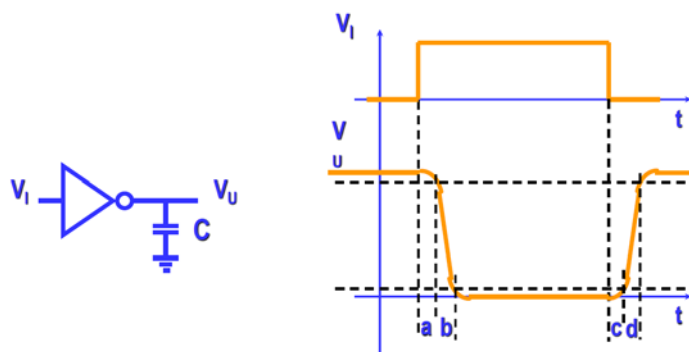
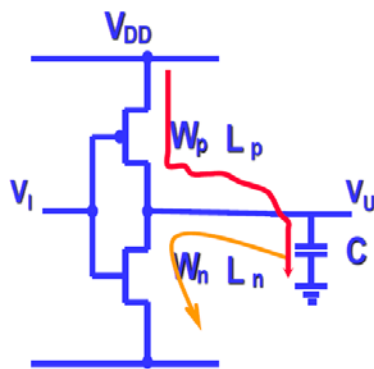


Figure 8- Caricare una capacità.

Ritardo di propagazione dell'inverter



$$t_{pHL} \propto \frac{KC}{\beta_n} \frac{1}{V_{DD} - V_{Tn}}$$

$$t_{pLH} \propto \frac{KC}{\beta_p} \frac{1}{V_{DD} + V_{Tp}}$$

$$\beta_n = \mu_n C_{ox} \frac{W_n}{L_n} \quad \beta_p = \mu_p C_{ox} \frac{W_p}{L_p}$$

Figure 9 - Calcolo del ritardo di propagazione in funzione delle caratteristiche dell'inverter.

Consumo di Potenza

- ✧ **Charge:** Capacitor store energy $E_C = \frac{1}{2} CV^2$
- ✧ **Power Supply** provides energy $E_f = CV^2$
- ✧ **Discharge:** Capacitor gives its energy to the inverter
- ✧ In one cycle the inverter consumes an energy equal to $E_D = CV^2$
- ✧ In the time T the inverter consumes a power
consumption of $P_D = E_D/T = P_D = CV^2/T = CV^2f$

Processo di fabbricazione dei CMOS

La fabbricazione dei transistor CMOS parte dai "wafer" di silicio.

Litografia

Processo simile alla stampante 3D. Ad ogni passo, vengono depositati materiali diversi. Strato dopo strato il circuito viene alla luce.

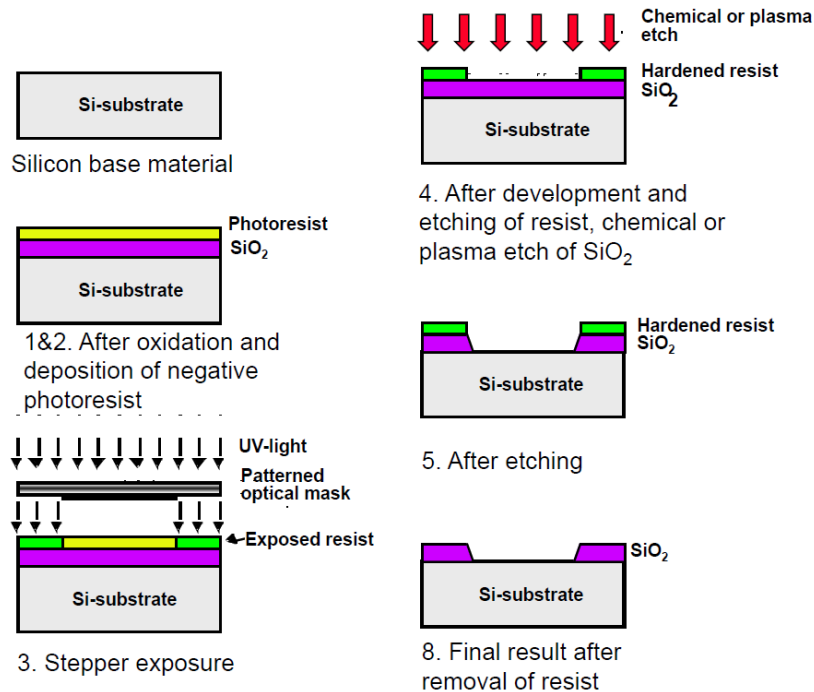


Figure 10 - Litografia.

Nello step 3 la luce UV serve per esporre specifiche aree ad una certa reazione chimica che le rende più facili da rimuovere.

Photolithography

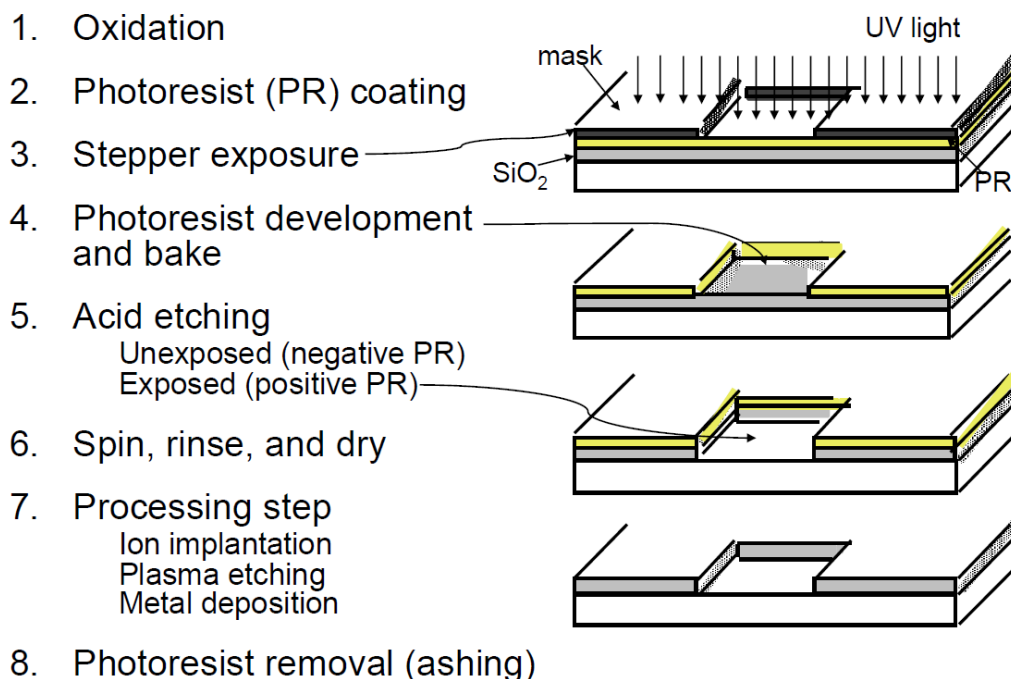


Figure 11- Processo della Photolithography.

1. Pattern masking
(photolithography)
2. Deposit material over
entire wafer
CVD (Si_3N_4)
chemical deposition
(polysilicon)
sputtering (Al)
3. Etch away unwanted
material
wet etching
dry (plasma) etching

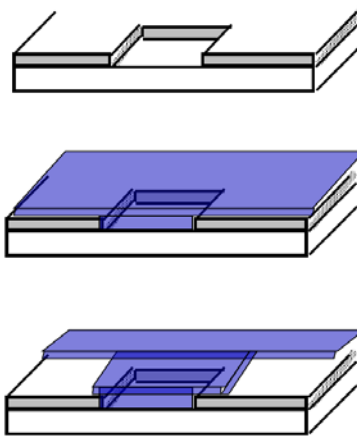


Figure 12 - In questa figura si nota come funziona il processo base: prima di deposita il materiale sopra la maschera e poi si incide.

Cross-section e layout dell'inverter

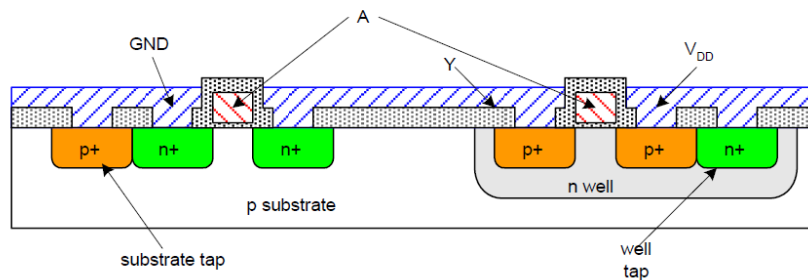


Figure 13- Cross-section dell'inverter.

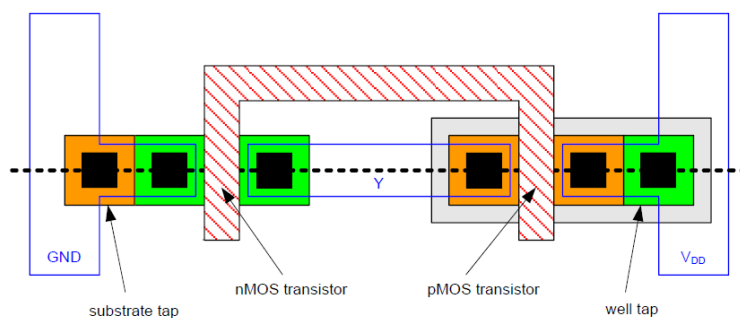


Figure 14 - Layout dell'inverter

Fabbricazione di circuiti integrati

Vediamo ora come si crea un inverter passo dopo passo:

L'inverter verrà costruito dal basso verso l'alto.

Wafer

Si inizia con un wafer di silicio (detto anche "blank wafer") esso è come una tela bianca nella quale incideremo il nostro dispositivo. Questo viene drogato in modo da farlo diventare carico di lacune (tipo P) esso sarà il nostro substrato.



Figure 15- Creazione del substrato di tipo P.

- First step will be to form the n-well:
 - o Remove layer where n-well should be built
 - o Implant or diffuse n dopants into exposed wafer
 - o Strip off SiO₂

Ossidazione

Si copre il wafer con un ossido protettivo (SiO₂).

Il processo è eseguito a 900 –1200 °C con acqua o ossigeno in una apposita fornace di ossidazione.



Figure 16 - L'ossido ricopre il substrato di tipo P.

Photoresist

Sopra l'ossido viene messo un polimero organico sensibile alla luce chiamato "Photoresist".

Questo materiale si ammorbidisce quando è esposto alla luce.

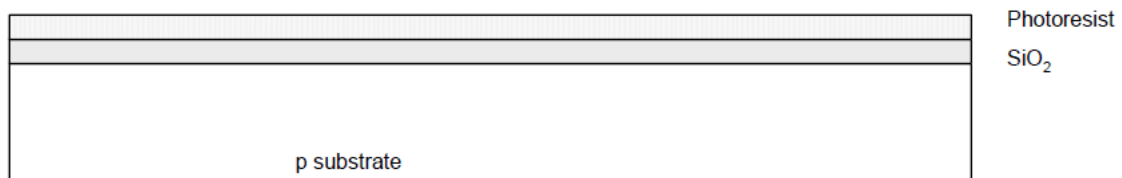


Figure 17- Il photoresist ricopre l'ossido.

Litografia

Una parte del Photoresist viene incisa per fare spazio ad una maschera di silicio drogata (tipo N) chiamata "maschera".

La maschera serve solo a prendere le misure.

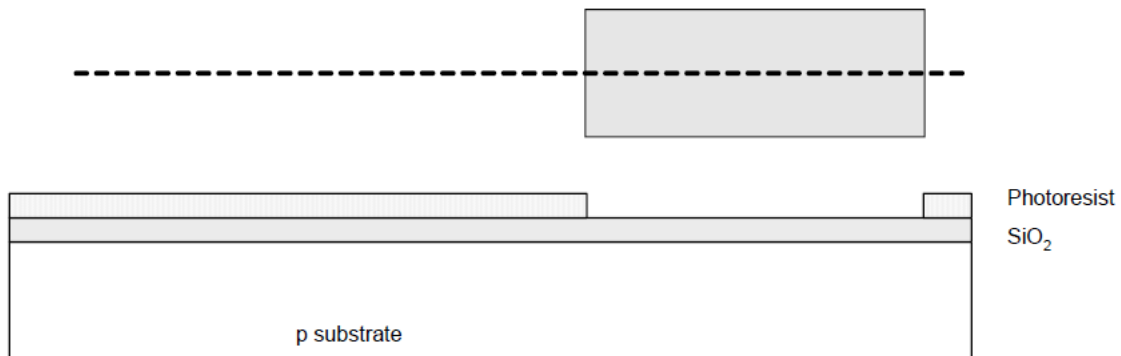


Figure 18 - La maschera viene adagiata sopra l'ossido dopo aver fatto spazio tra il photoresist.

Incisione

Tramite acido fluoridrico si incide l'ossido. L'acido non attacca il photoresist; quindi, solo la parte esposta dell'ossido viene incisa.

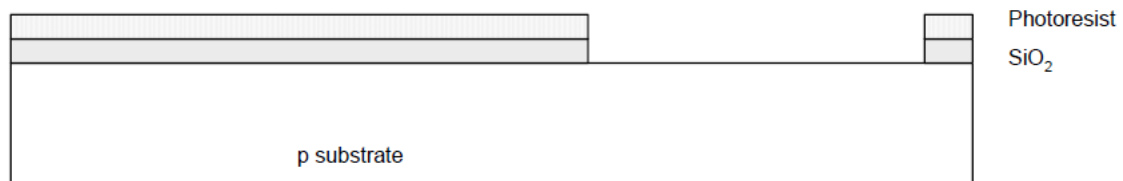


Figure 19- Incisione dall'ossido.

Rimozione del photoresist

Usando una mix di acidi (chiamato piranha) si rimuove il photoresist.



Figure 20- Il piranha morde come un pazzo furioso il photoresist e lo ammazza male.

Inserimento dello "n-well"

n-well è la parte di ossido drogata N e viene applicata tramite diffusione o "Ion implantation".

Diffusione

Si piazza il wafer tramite gas arsenico, il quale carica la parte esposta di elettroni (drogaggio di tipo N).

Ion Implantation

Si spara contro al wafer un raggio di ioni, i quali rimangono entrano e rimangono bloccati nella parte di silicio esposta.

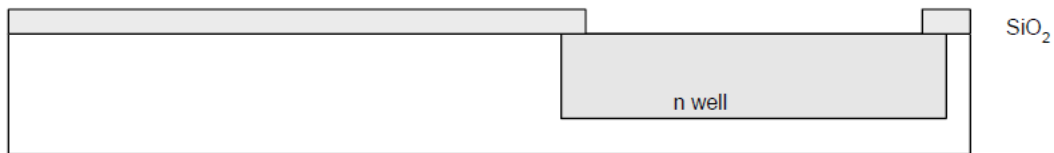


Figure 21- Risultato finale della diffusione o Ion Implantation

Rimozione dell'ossido

Usando dell'acido fluoridrico si rimuove il resto dell'ossido.



Figure 22 - Dopo aver rimosso l'ossido.

Inserimento polisilicio

Si deposita uno strato sottilissimo di poli silicio, il quale fungerà da gate del transistor.

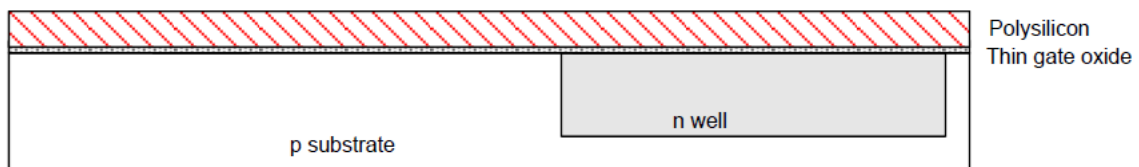


Figure 23- Dopo aver messo i polisilicio.

Litografia del polisilicio

Tramite il processo di Litografia, si ottengono 2 pin, uno attaccato alla parte con drogaggio P e l'altra con drogaggio N.

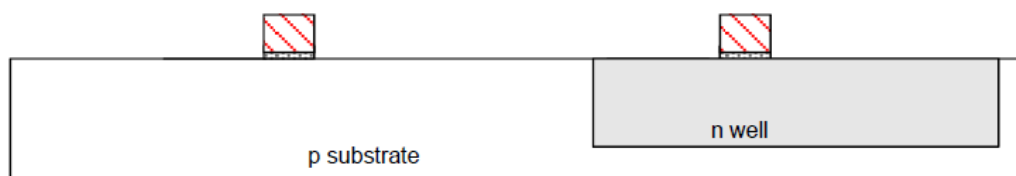


Figure 24 - Litografia del polisilicio.

Self-Aligned Process

Si ricopre il tutto con dell'ossido.

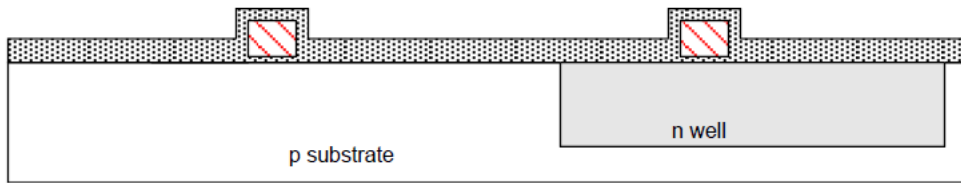


Figure 25 – L'Ossido ricopre la superficie.

N-diffusion

Si incidono sull'ossiduo 3 aree nelle quali andrò (come prima) a creare zone di drogaggio di tipo N+.

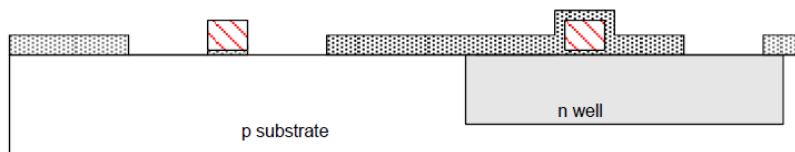


Figure 26 - Dopo aver inciso l'ossido n+.

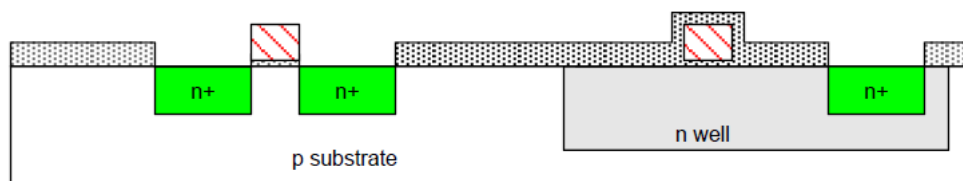


Figure 27- Dopo il processo di diffusione.

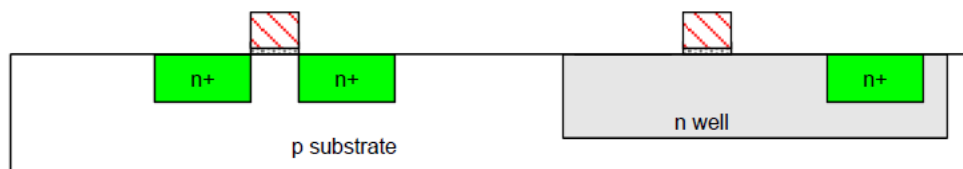


Figure 28 - Dopo aver rimosso l'ossido.

P-Diffusion

Identico alla N-diffusion, ma ora si droga con un drogaggio di tipo P.

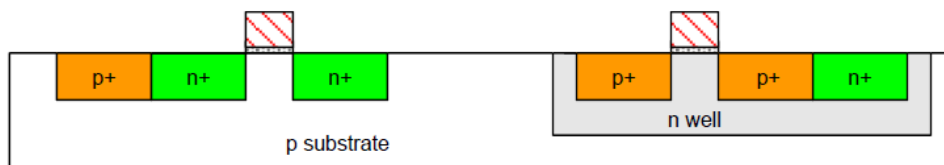


Figure 29- Dopo la P-diffusion.

Contatti

Si ricopre tutto con uno spesso strato di ossido, poi si incide per fare spazio ai contatti.

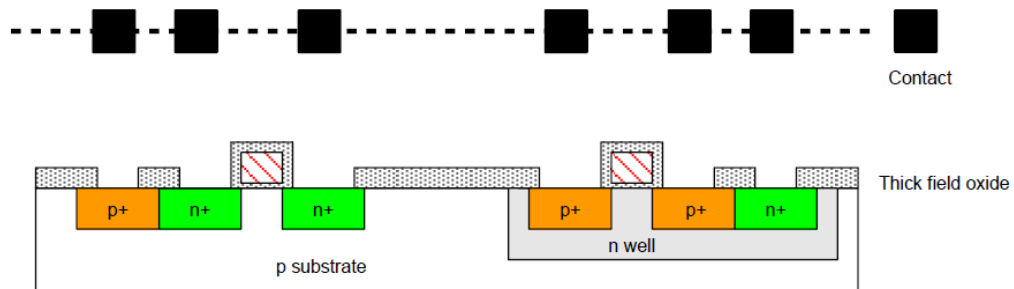


Figure 30 - Dove piazzerò i contatti.

Metallizzazione

Ricopro di alluminio l'intero wafer.

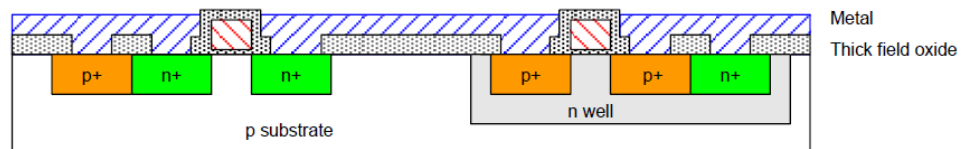


Figure 31- Metallizzazione.

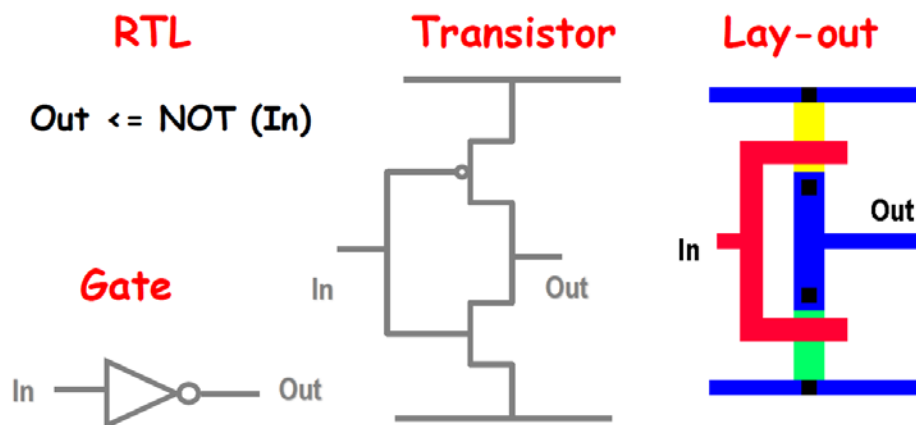
Regole di design

- Interface between the circuit designer and process engineer;
- Guidelines for constructing process masks;
- Unit dimension: minimum line width:
 - scalable design rules: lambda parameter
 - absolute dimensions: micron rules
- Rules constructed to ensure that design works even when small fab errors (within some tolerance) occur;
- A complete set includes:
 - set of layers
 - intra-layer: relations between objects in the same layer
 - inter-layer: relations between objects on different layers

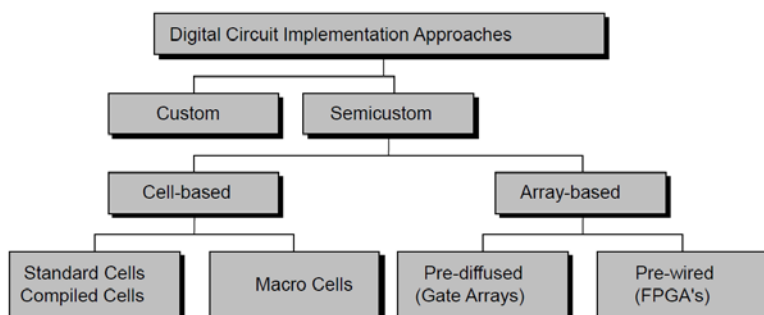
Why Have Design Rules?

- To be able to tolerate some level of fabrication errors such as:
 - Mask misalignment
 - Dust
 - Process parameters (e.g., lateral diffusion)
 - Rough surfaces

Electronic abstraction levels



Integrated circuit design style

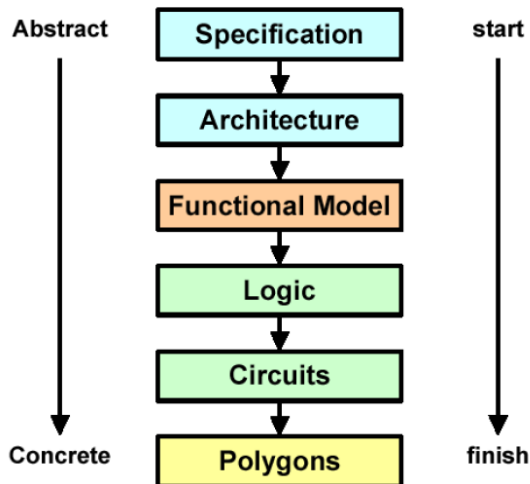


Full-Custom Design

It gives the maximum freedom in terms of sizing, placement, and interconnection for each single transistor.

- Maximum speed and minimum area occupancy.
- High cost and development time.
- High skills are required on ASIC technology, electronic circuit, and architecture.

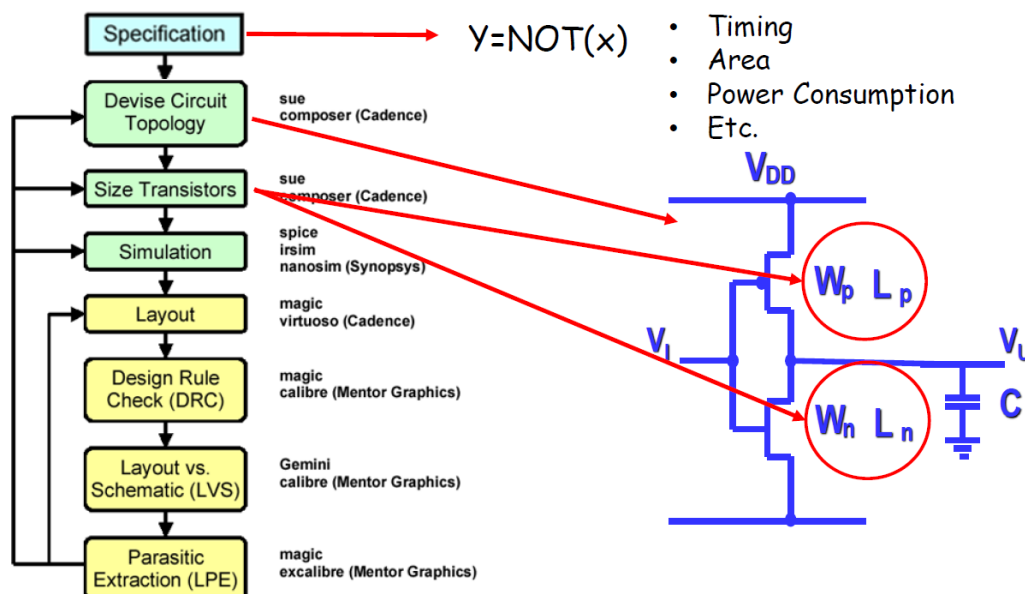
There are different levels of design abstraction, we consider the entire design at top levels only, and while we go down in the stack it increases the amount of the automation.



Let's consider an inverter, we got some requirements in terms of area and power consumption.

Start with full custom design:

1. First, we define the true table of the inverter, in order to create the initial topology of the inverter.
2. At this point we have an idea in terms of area, but we have no idea about the wires that connect all the parts of the inverter.
3. Then we have to simulate the model, checking if it is right.
 - a. We check with waveforms.
4. Now we have to define the layout, in which we define how to create the PMOS and NMOS with precise values before we extract this information.
5. Now we have to do the design rule heck that checks all the distances between components.



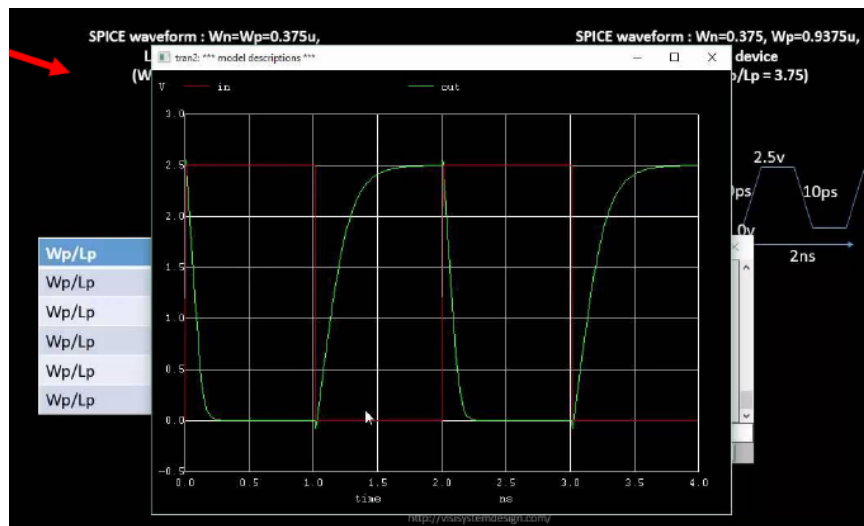


Figure 32- Example of waveform

Layout

I progettisti traducono il progetto schematico in un layout fisico dettagliato, che definisce la posizione e le dimensioni esatte di ogni componente del circuito, nonché le **interconnessioni** tra di essi. Questa fase coinvolge l'utilizzo di strumenti di layout CAD per posizionare e collegare i componenti in modo ottimale.

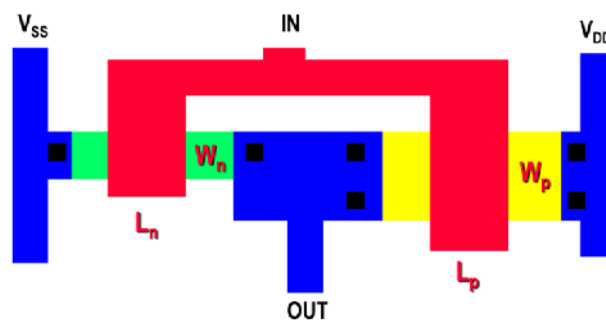


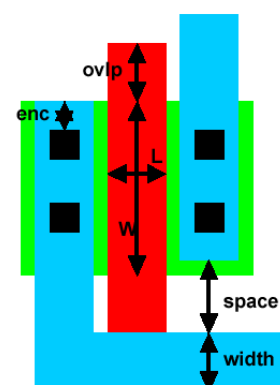
Figure 33- Example of layout.

Design Rule Check [DRC]

Nel contesto del design di circuiti integrati (IC), la Design Rule Check (DRC) è un processo fondamentale nella fase di progettazione. **La DRC è un controllo automatico eseguito da software specializzati per verificare che il layout del circuito soddisfi le regole di progettazione predefinite**, chiamate regole di progettazione. Queste regole sono stabilite in base alle capacità del processo di produzione utilizzato per fabbricare il circuito integrato.

Regole di progettazione

Le regole di progettazione possono riguardare parametri come le dimensioni delle strutture del circuito, le distanze minime tra le linee, le aree di contatto, le proporzioni dei dispositivi elettronici, le interconnessioni e molti altri aspetti del layout del circuito. **L'obiettivo della DRC è assicurare che il layout del circuito sia conforme a queste regole, in modo da evitare problemi di produzione, quali cortocircuiti, sovrapposizioni, problemi di isolamento e altre violazioni che potrebbero compromettere il funzionamento del circuito o la sua producibilità.**



CAD

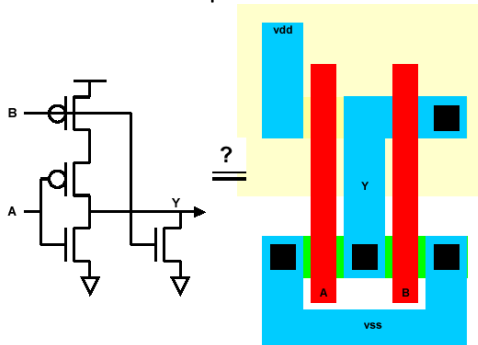
La DRC viene solitamente eseguita in modo automatico tramite software di progettazione assistita da computer (CAD) specificamente progettati per il design dei circuiti integrati.

Layout vs Schematic [LVS]

Una volta completato il layout, un software specializzato confronta il layout fisico con il progetto schematico per assicurarsi che corrispondano esattamente.

Risoluzione dei problemi: Se vengono rilevate discrepanze durante la verifica LVS, è necessario risolverle per garantire che il layout sia corretto e funzionante. Questo potrebbe richiedere modifiche al layout stesso o al progetto schematico, a seconda del tipo di discrepanza.

Il processo LVS è cruciale nel garantire che il circuito integrato progettato funzioni correttamente una volta fabbricato. Assicura che non vi siano errori nel layout che potrebbero compromettere le prestazioni o la funzionalità del dispositivo finale.



Layout Parasitic Extraction [LPE]

Si concentra sull'estrazione e sulla modellazione dei parametri parassitici presenti nel layout fisico del circuito. Questi parametri parassitici possono includere capacità, induttanze, resistenze e altre caratteristiche che influenzano le prestazioni e il comportamento del circuito. Consente ai progettisti di valutare e ottimizzare le prestazioni del circuito tenendo conto di tali effetti.

Semi-Custom Design

Approccio intermedio tra la progettazione full custom e la progettazione standard cell (cellule standard), ed è ampiamente utilizzata nell'industria dei circuiti integrati per ottimizzare i tempi di sviluppo e i costi di progettazione. **Le fonderie forniscono blocchi preconfezionati e generici che noi utilizziamo per creare circuiti più grossi.**

Gate Array Structure

Questo approccio combina elementi di personalizzazione limitata con una vasta disponibilità di celle logiche predefinite, consentendo una progettazione più rapida ed economica rispetto alla progettazione full custom. La Gate Array Structure si basa sulle matrici di porte logiche:

Matrici di porte logiche

Matrice regolare di celle logiche ripetute su tutta l'area del chip. **Queste celle logiche possono includere porte logiche, flip-flop, latch e altri elementi di base utilizzati nella costruzione di circuiti digitali.**

Le connessioni tra le celle logiche sono predefinite e fissate durante la fase di progettazione della struttura a gate array.

La fabbrica confeziona questo insieme di porte logiche e che poi i progettisti collegano tramite canali.

Personalizzazione limitata

Nonostante la presenza di una vasta matrice di celle logiche, la personalizzazione all'interno di una struttura a gate array è limitata. I progettisti possono personalizzare solo una piccola parte del chip, nota come "zona di personalizzazione" o "zona di interconnessione". In questa zona, i progettisti possono creare circuiti più complessi, utilizzando interconnessioni personalizzate tra le celle logiche.

Vantaggi della Gate Array Structure

- **Costi ridotti:** La disponibilità di una vasta gamma di celle logiche predefinite riduce i costi di progettazione e produzione, in quanto non è necessario progettare ogni componente da zero.
- **Velocità di sviluppo:** Grazie alla struttura regolare e alle celle logiche predefinite, la progettazione di un chip basato su gate array è più rapida rispetto alla progettazione full custom.
- **Flessibilità limitata:** Sebbene la personalizzazione sia limitata rispetto alla progettazione full custom, la presenza di una zona di interconnessione consente comunque una certa flessibilità nella progettazione di circuiti specifici.

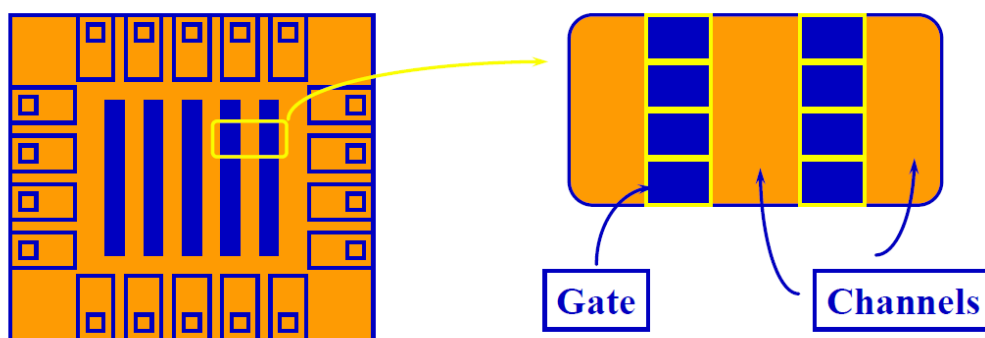


Figure 34 - Esempio di Gate Array- le parti blu sono le interconnessioni.

Standard Cell Structure

Fornisce una selezione più limitata di celle logiche predefinite, ma offre maggiore flessibilità e prestazioni rispetto alla gate array.

Librerie di celle standard

La caratteristica principale di una struttura a cella standard è l'uso di librerie di celle standard predefinite. Queste librerie contengono una vasta selezione di blocchi funzionali, come porte logiche, flip-flop, latch e altre componenti di base utilizzate nella costruzione di circuiti digitali. Le celle in queste librerie sono progettate per essere facilmente configurabili e riutilizzabili in diverse applicazioni di progettazione.

Personalizzazione delle interconnessioni

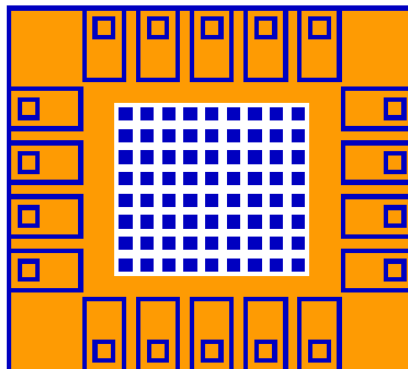
A differenza della gate array, in cui le interconnessioni sono predefinite e fisse, nella struttura a cella standard, si ha maggiore flessibilità nella personalizzazione delle interconnessioni tra le celle. Ciò consente la creazione di circuiti più complessi e ottimizzati per le prestazioni, poiché le interconnessioni possono essere progettate in base alle specifiche esigenze del circuito.

Vantaggi della Standard Cell Structure

- **Flessibilità:** I progettisti possono selezionare e configurare le celle standard in modo da soddisfare le esigenze specifiche del circuito.
- **Prestazioni ottimizzate:** Poiché i progettisti hanno maggiore controllo sulle interconnessioni tra le celle, è possibile ottimizzare le prestazioni del circuito, riducendo la propagazione del segnale, minimizzando i ritardi e migliorando l'efficienza complessiva.

Svantaggi della Standard Cell Structure

- **Velocità di sviluppo:** Richiede più tempo rispetto alla gate array.



ASIC (Application-Specific Integrated Circuit)

Circuito creato appositamente per risolvere un problema: consente di raggiungere delle prestazioni in termini di velocità di processamento e consumo elettrico difficilmente ottenibili con l'uso di soluzioni più generiche (general purpose).

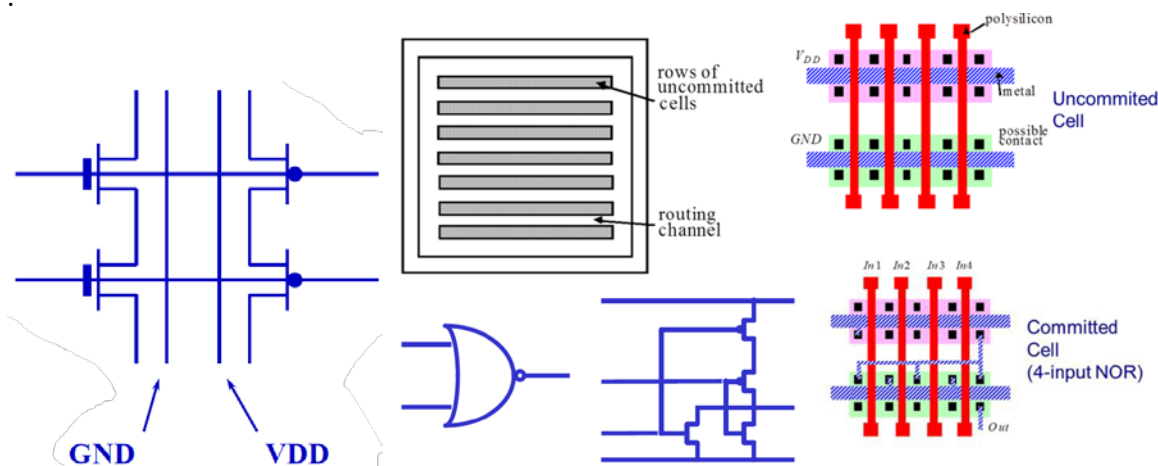
Sviluppo molto costoso e per questo motivo il loro impiego è limitato a campi in cui possano essere usati in maniera massiccia, ovvero alti volumi di mercato, come l'elettronica di consumo.

Mentre per usi su scala più limitata vengono preferite, ad esempio, tecnologie riprogrammabili come le FPGA. Uno specifico settore in cui si sono fatti strada i processori ASIC è il mondo delle criptovalute, grazie alle elevate prestazioni raggiungibili con questo tipo di tecnologia.

Single gate Structure

L'"ASIC Single Gate Structure" (Struttura a Porta Singola ASIC) si riferisce a un tipo specifico di circuito integrato programmabile progettato per implementare una singola funzione logica o un singolo gate logico. Questo tipo di struttura è spesso utilizzato in applicazioni in cui è necessaria una piccola quantità di logica programmabile o una funzione logica specifica.

È composta da 2 NMOS e da 2 PMOS e da un'interconnessione dal channel al drain, source al gate del transistor.



Gate array interconnection

Connessione di una matrice di porte logiche programmabili (gate array) all'interno di un circuito integrato applicativo specifico (ASIC). In una struttura di gate array ASIC, una vasta matrice di porte logiche (come porte AND, OR, NOT, ecc.) è predisposta su un chip in un modello regolare.

Matrice di porte Logiche programmabili

La base di una ASIC gate array è una matrice di porte logiche. Queste porte sono generalmente organizzate in una griglia regolare e possono essere configurate per eseguire varie funzioni logiche.

Interconnessione programmabile

Le porte logiche all'interno della matrice sono interconnesse attraverso una rete di collegamenti programmabili. Questi collegamenti possono essere configurati in modo da collegare le porte logiche in modalità specifiche per implementare la funzionalità desiderata.

Gli ingegneri possono configurare i collegamenti tra le porte logiche in modo da soddisfare le specifiche dell'applicazione. Questo offre un alto grado di flessibilità nella progettazione di circuiti personalizzati senza dover progettare e fabbricare un circuito integrato da zero.

Efficienza e Flessibilità

L'utilizzo di una matrice di porte logiche programmabili offre un compromesso tra l'efficienza di una soluzione ASIC specificamente progettata e la flessibilità di una FPGA (*Field-Programmable Gate Array*). È possibile ottenere prestazioni ottimizzate per l'applicazione specifica senza sacrificare la capacità di personalizzazione.

In conclusione, la "ASIC Gate Array Interconnection" si riferisce alla configurazione programmabile delle interconnessioni tra le porte logiche all'interno di una matrice di porte logiche su un chip ASIC.

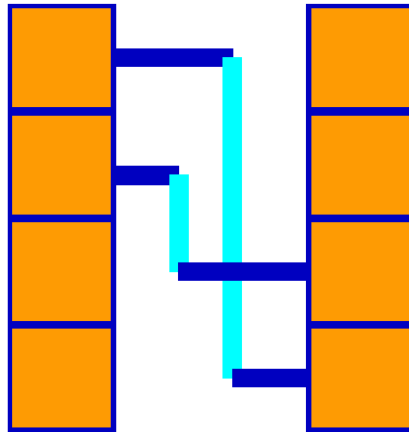


Figure 35- Interconnessioni tra porte logiche di due matrici.

Sea-of-gate structure

Si basa su una matrice regolare di celle di base contenenti porte logiche. Prevede la disposizione di un gran numero di celle di base all'interno di una griglia su un chip.

Matrice Regolare

La caratteristica principale di una struttura Sea-of-Gates è la sua disposizione regolare. Le celle di base, ognuna contenente una o più porte logiche, sono organizzate in una griglia uniforme su tutto il chip.

Interconnessione Programmabile

Le celle di base sono collegate tra loro attraverso una rete di interconnessione programmabile. Questo consente agli ingegneri di configurare le connessioni tra le porte logiche per implementare la logica desiderata per l'applicazione specifica e in modo da soddisfare le specifiche dell'applicazione, consentendo la creazione di circuiti integrati altamente personalizzati.

Efficienza

La disposizione regolare delle celle di base e la possibilità di condividere le risorse tra di esse può portare a una maggiore efficienza in termini di area di chip, consumo di energia e prestazioni rispetto ad altre soluzioni di progettazione ASIC.

Applicazioni

La struttura Sea-of-Gates è ampiamente utilizzata in una vasta gamma di applicazioni, inclusi dispositivi elettronici di consumo, apparecchiature di rete, sistemi embedded, dispositivi di elaborazione di segnali digitali, e altro ancora. È particolarmente adatta per applicazioni in cui è richiesta una combinazione di prestazioni ottimizzate e flessibilità di progettazione.

Register Transfer Level [RTL]

We'll describe in Verilog what is happening in every edge of the clock.

The path that is characterized by the greater delay is the one that limit the minimum clock period.

Functional Verification

In Verilog there are some possible problems:

- I can write something that is not standard.
 - o This is not a big problem; the Verilog compiler can warn me about errors or not standard things.
- Write something that does not represent the functionality that I want.
 - o Debugging: Test of the digital design.
 - o So, I must prepare some test cases for assure the functionality of the digital design.
 - o I write another file that contain a testbench which reproduce the input of the design under test and check the output.
 - o But before write the testbench I must define a test plan, so I must think at every test I need to do.

Logic Synthesis

Processo compiuto da software specializzati, i quali richiedono:

- Descrizione VHDL del circuito.
- Librerie di tecnologie che voglio usare nel circuito.
- Requisiti di design e obiettivi, come ad esempio:
 - $\text{Area} < X \text{ pm}$
 - $\text{Power Consumption} < Y \text{ mW}$
 - $F_{\text{max}} (\text{frequenza massima di clock}) > Z \text{ MHz}$

Non è sicuro che il prodotto del software rispetterà tutti i vincoli, questo perché non c'è alcun controllo sull'effettiva fattibilità del circuito. Il tool potrebbe anche accettare vincoli impossibili (tipo $\text{Area} = 0$).

Place and Route

In questo step, il circuito sintetizzato (*quindi dopo la sintesi logica fatta dal software*) viene mappato fisicamente su un chip, dove vengono assegnate le posizioni fisiche ai componenti del circuito e vengono tracciate le interconnessioni tra di essi. Questo processo è fondamentale per garantire che il circuito funzioni correttamente e rispetti i requisiti di performance, consumo energetico e layout.

La fase di Place and Route è essenziale per garantire che il circuito integrato funzioni correttamente e soddisfi i requisiti di performance, consumo energetico e area. Un'efficace Place and Route può influenzare significativamente le prestazioni e la producibilità del chip finale.

Costo di un circuito integrato

Una delle metriche da considerare ovviamente.

$$\text{Costo Totale} = \text{NRE} + (N * \text{RE})$$

Non-Recurrent engineering Cost [NRE]

Sono i costi di design e progettazione. Sono costi indipendenti dal numero di unità prodotte.

- Licenza dei tool usati.
- Stipendi.
- Costo dovuto allo stampo per produrre i circuiti.
- Tool di test per gli ASIC.

Recurrent Engineering Cost [RE]

Sono i costi di produzione. Quindi dipendenti dal numero di unità prodotte.

- **Materie prime**
 - o Il wafer di silicio.
 - o Altri metalli.
- **Processo di produzione**
 - o Acqua, elettricità e manutenzione.
 - o Di solito proporzionale alla grandezza del chip voluto.
- **Package**
 - o Il box che conterrà il chip.
- **Test**
 - o Ogni singolo circuito in uscita va testato con delle grosse e costose macchine, ovviamente le nolegiamo; quindi, le paghiamo per unità di tempo.
 - o Di solito per testare un circuito ci si mette qualche secondo.

Costo totale di produzione

Consideriamo che produrremo N unità.

$$C_t = \text{NRE} + n * \text{RE}$$

Costo per unità

$$C_1 = (C_t / n) = (\text{RE} + \text{NRE}/n)$$

$$C_1 \geq \text{RE} \text{ SEMPRE.}$$

Come risparmiare

Miglior stile di design con N piccolo? Gate-array, Poiché ha NRE bassi.

Performa bene con N basso perché non posso ottimizzare l'area.

Stile di design che richiede meno area? Full-Custom, perchè ottimizzo tutto però conviene solo a fronte di N molto grande (*posso ammortizzare*), poiché NRE è altissimo.

FPGA (Field Programmable Gate-Array unit)

Dispositivo di circuito integrato che può essere programmato e configurato dopo la sua produzione. Contrariamente ai circuiti integrati tradizionali, come ASIC (Application-Specific Integrated Circuit), che sono fabbricati per uno scopo specifico durante la produzione e non possono essere modificati, **gli FPGA possono essere programmabili per svolgere varie funzioni specifiche dopo essere stati fabbricati.**

Da cosa sono fatti?

Gli FPGA sono costituiti da una matrice di blocchi logici configurabili (*come porte logiche, latch e flip-flop*) e interconnessioni programmabili che permettono agli utenti di implementare qualsiasi funzione logica desiderata. Possono essere programmati utilizzando linguaggi di descrizione hardware come VHDL.

Vantaggi di FPGA

I vantaggi principali degli FPGA includono la flessibilità e la riconfigurabilità, che li rendono adatti a una vasta gamma di applicazioni, dalla prototipazione rapida alla produzione di dispositivi su larga scala.

Programmare un FPGA

FPGA SRAM Base (Xilinx)

Lo stile di sviluppo SRAM-based (Static Random-Access Memory) per FPGA si riferisce a un tipo di tecnologia utilizzata per la realizzazione di FPGA, in cui **la configurazione del dispositivo è memorizzata all'interno di celle di memoria statica volatile (SRAM).**

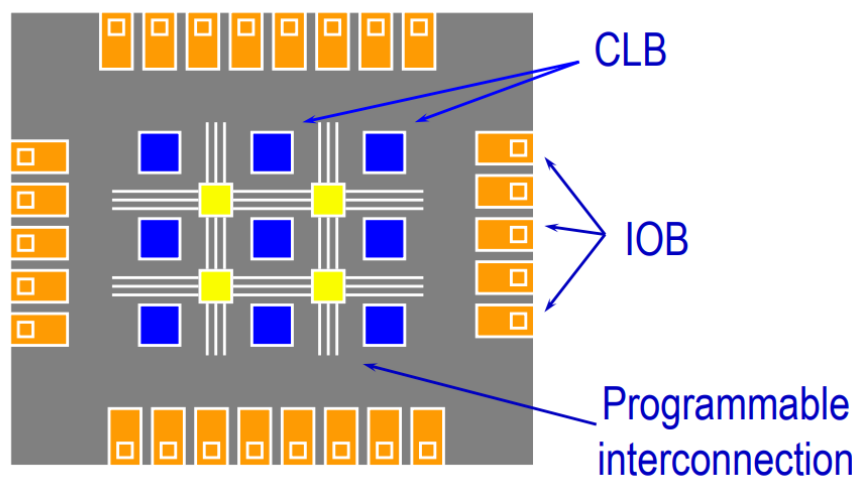


Figure 36- FPGA Block.

CLB (Configurable Logic Block)

Blocco hardware dove sono implementate delle funzioni logiche. Ogni CLB è una rete sequenziale sincronizzata con clock, enable ed elementi di memoria. Ogni CLB ha anche una PROM (*la quale permette di implementare qualsiasi funzione logica*).

IOB (Input-Output-Circuits)

Interfacce I/O programmabili.

Configurazione temporanea (Programmable interconnection)

La configurazione del dispositivo viene caricata da un dispositivo di memoria esterna (*come un chip di memoria SRAM*) ogni volta che il dispositivo viene avviato o reset, consentendo una maggiore flessibilità e programmabilità rispetto ad altre tecnologie di FPGA. La configurazione del dispositivo FPGA viene memorizzata temporaneamente in una memoria SRAM esterna o all'interno del dispositivo FPGA stesso.

General interconnection

Vengono utilizzati gli switch (chiamati anche *exchange matrix*) del board.

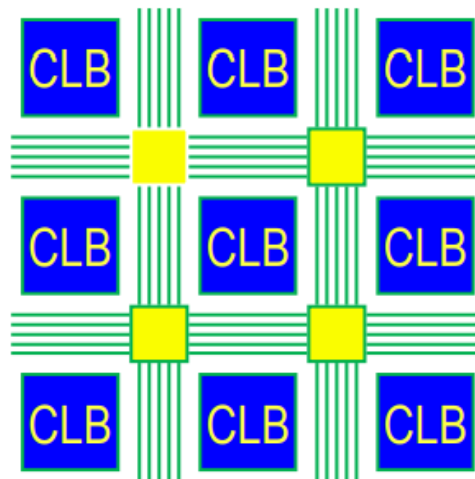


Figure 37- Situazione iniziale, nei blocchi gialli (*exchange matrix*) verranno implementate le interconnessioni.

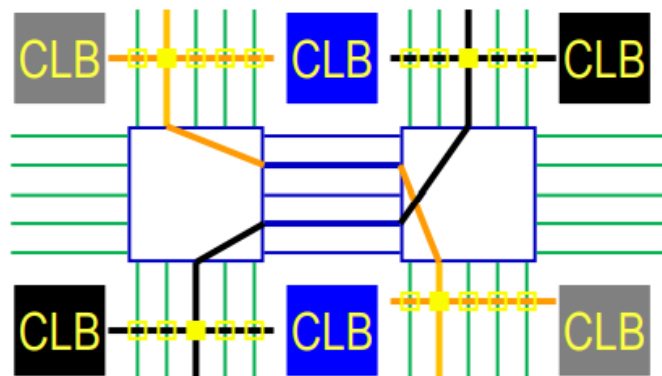


Figure 38- Situazione finale, le interconnessioni sono state programmate.

Direct interconnection

Connessioni dedicate tra CLB adiacenti. Permette di avere ritardo di propagazione minimo e spesso anche calcolabile a priori. Non usa risorse generali.

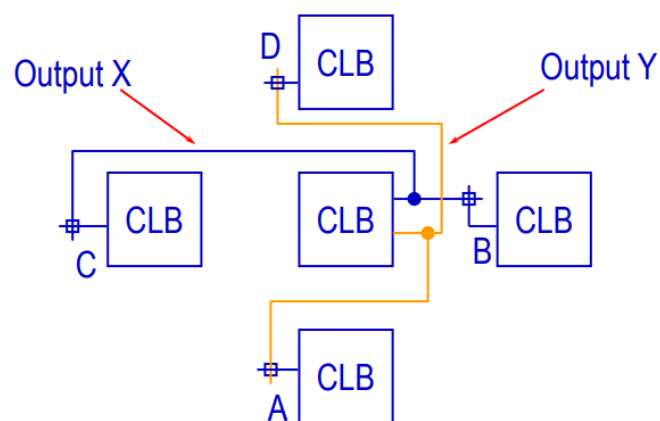


Figure 39 - Connessione diretta.

Global interconnection

Vengono definite lunghe linee che attraversano tutto il circuito senza usare delle exchange Matrix. Ho il minimo ritardo di propagazione del clock (quindi è il modo migliore per rendere i CLB sincronizzati).

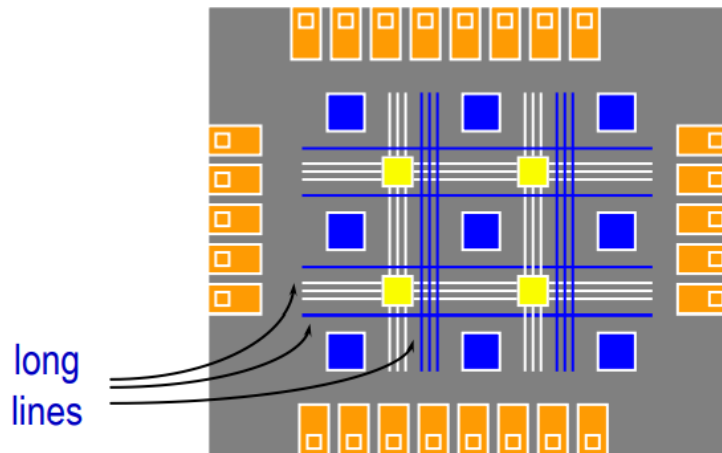


Figure 40- Global interconnection.

Flessibilità

Poiché la configurazione è memorizzata in una memoria esterna, può essere modificata o riscritta più volte senza danneggiare il dispositivo FPGA stesso. Questo consente una maggiore flessibilità nel processo di sviluppo e iterazione del progetto.

Inoltre, alcuni FPGA possono essere riconfigurati dinamicamente durante l'esecuzione, consentendo di adattare il dispositivo alle esigenze specifiche dell'applicazione senza dover interrompere il funzionamento del sistema.

Facilità di sviluppo

Questo stile di sviluppo semplifica il processo di progettazione e sviluppo, poiché consente ai progettisti di testare rapidamente diverse configurazioni e iterare sul design senza dover ricorrere a costosi processi di produzione di chip personalizzati.

FPGA basato su tecnologie anti-fused

La configurazione del dispositivo è memorizzata in modo permanente all'interno di cellule di memoria che possono essere programmate attraverso la "fusione" o la "antifusione" di determinati collegamenti.

Configurazione permanente

Una volta che le connessioni sono fuse, la configurazione del dispositivo FPGA diventa permanente e non può essere modificata. Questo differisce dagli FPGA SRAM-based, dove la configurazione può essere modificata dinamicamente.

Robustezza e sicurezza

Offre un'elevata sicurezza e robustezza contro tentativi di hacking o manipolazioni non autorizzate, poiché non è possibile modificare la configurazione una volta che è stata impostata.

Riassunto degli stili di design dei circuiti integrati

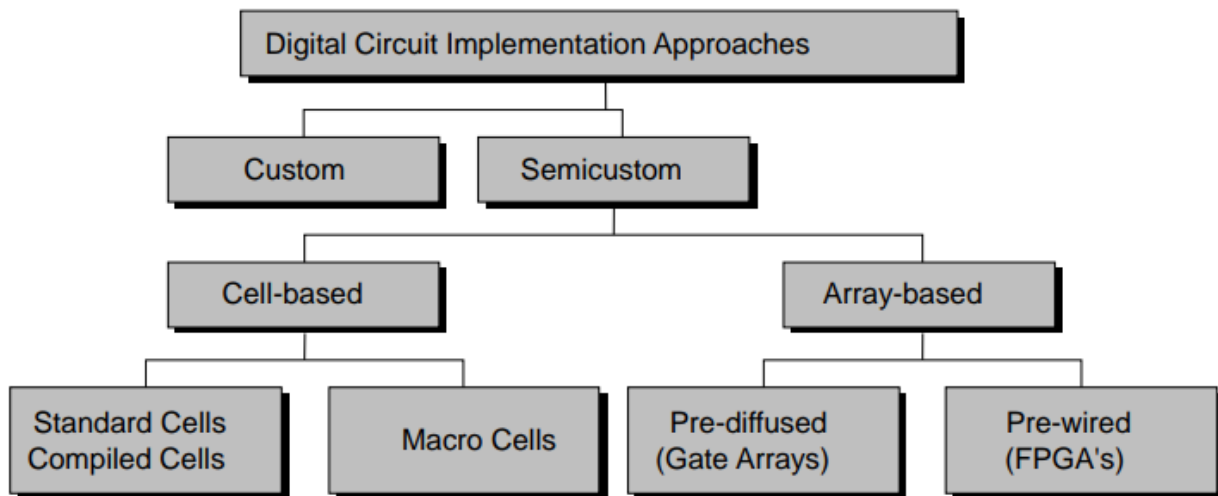


Figure 41- Stili di design divisi per famiglie.

The “Energy-Flexibility” Gap

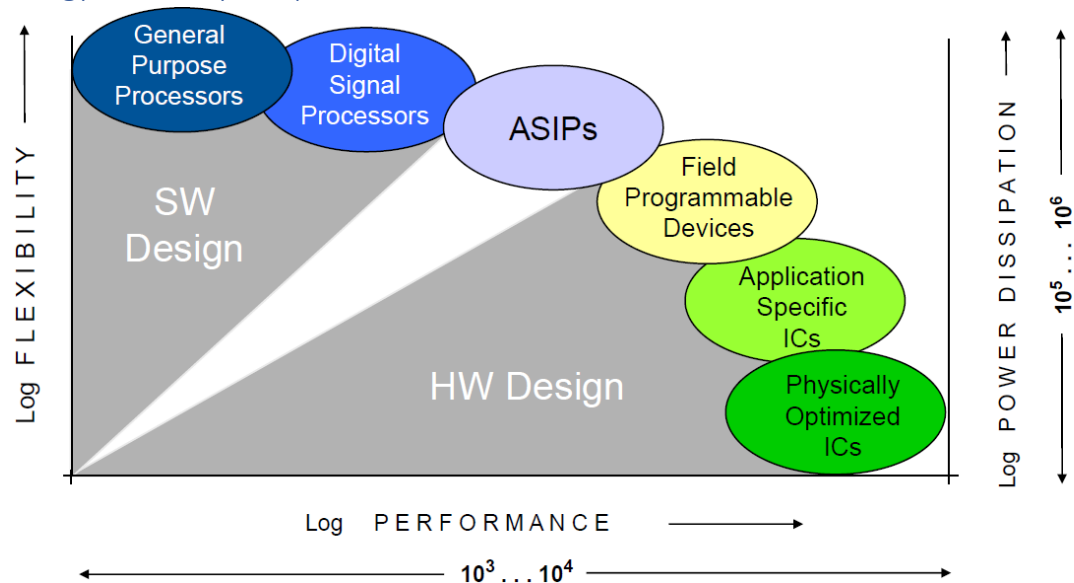


Figure 42- Trade-off di ogni possibile stile di design.

Come si può vedere, la full custom design offre la maggiore ottimizzazione di area e performance ma molta meno flessibilità. Invece FPGA (field-programmable design) presenta minore capacità di performance ma più flessibilità. Il Massimo della flessibilità la si ottiene da un processore general purpose, il quale ha la capacità di eseguire software e quindi potenzialmente fare qualsiasi cosa; però è la soluzione più lenta e ingombrante.

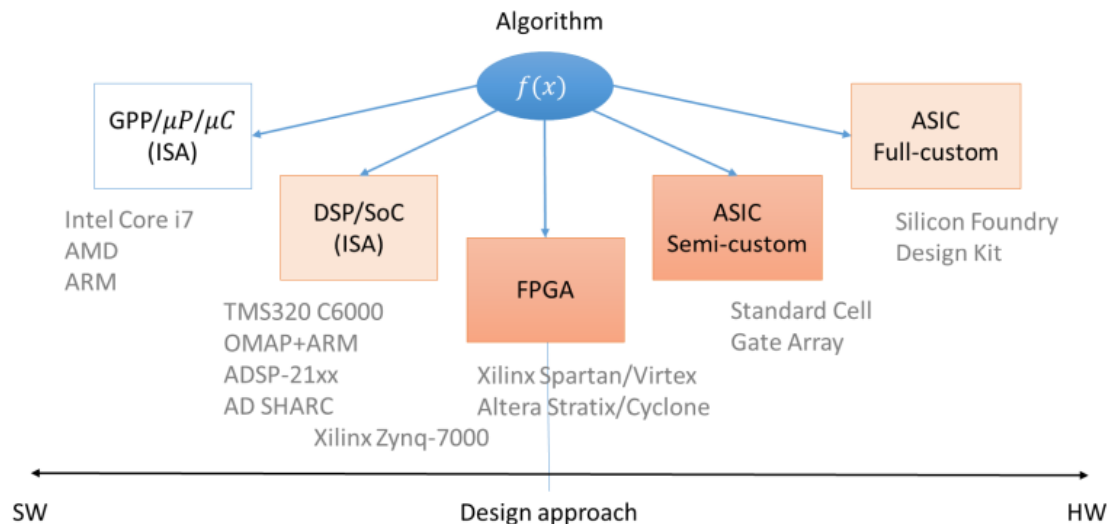


Figure 43- Filosofia di ogni stile di design.

SoC (System-on-Chip)

Gli elementi SoC (System-on-Chip) sono circuiti integrati che integrano tutti o la maggior parte dei componenti di un sistema elettronico su un singolo chip. Questi componenti possono includere processori centrali (CPU), unità di elaborazione grafica (GPU), DSP (*Digital Signal Processor*), memoria, interfaccia di I/O, controller di periferiche, unità di elaborazione specializzate e altro ancora.

L'obiettivo degli SoC è quello di fornire una soluzione completa e altamente integrata per una vasta gamma di applicazioni, consentendo un design più compatto, efficiente ed economico dei dispositivi elettronici. Gli SoC sono ampiamente utilizzati in dispositivi quali smartphone, tablet, smartwatch, dispositivi IoT (Internet of Things), sistemi embedded.

Grazie alla loro elevata integrazione e alla capacità di ospitare una vasta gamma di funzionalità su un singolo chip, gli SoC consentono di ridurre i costi di produzione, migliorare le prestazioni e l'efficienza energetica, e semplificare lo sviluppo e la produzione di dispositivi elettronici complessi.

programmable	reconfigurable	fixed
DSP, μ C, RISC, GPP, ...	FPGA / eFPGA	ASIC
Compiler, Assembler, Linker, Simulator, Debugger	Gate-Level Synthesis Tools, Power Analysis Tools, HW-Simulator	

La foundry library

Libreria di elementi di base fornita dalla fabbrica di semiconduttori (*appunto la foundry*) che produce il chip FPGA. Questa libreria contiene una vasta gamma di componenti e blocchi di base che possono essere utilizzati dai progettisti durante lo sviluppo del circuito su un FPGA.

La library è il set di hardware di cui dispone il FPGA per emulare la descrizione VHDL.

Porte logiche

Include porte logiche elementari come AND, OR, NOT, NAND, NOR, XOR, e XNOR. Queste porte logiche sono gli elementi di base utilizzati per costruire circuiti più complessi.

Flip-flop e latch

Questi sono gli elementi di memoria utilizzati per memorizzare informazioni all'interno del circuito. Possono includere flip-flop di tipo D, JK, SR, e latch.

Multiplexer e demultiplexer

Questi componenti consentono la selezione e il routing di segnali all'interno del circuito.

Contatori e registri

Utilizzati per contare e memorizzare dati sequenziali all'interno del circuito.

Memorie RAM e ROM

Blocchi di memoria utilizzati per memorizzare dati in modo temporaneo o permanente.

Adattatori di interfaccia

Forniscono interfacciamento con altri componenti del sistema, come bus di comunicazione o interfacce di input/output.

Convertitori di segnale

Includono convertitori di segnale analogico-digitale (ADC) e digitale-analogico (DAC) utilizzati per interfacciare il circuito FPGA con il mondo esterno.

La foundry library fornisce una base essenziale per lo sviluppo di circuiti su FPGA, poiché offre una serie di componenti predefiniti e ottimizzati per le specifiche del chip FPGA in questione. I progettisti possono utilizzare questi componenti come mattoni di base per costruire circuiti più complessi, risparmiando tempo e sforzo nello sviluppo di soluzioni su misura.

Tipologie di Logica

Logica combinatoria

L'output corrente dipende solo dall'input corrente.

Logica sequenziale

L'output corrente dipende dall'input corrente e anche dagli input passati.

Asynchronous Sequential Logic

L'output corrente dipende dall'input corrente e anche dagli input passati, però l'output è trasparente, quindi se l'input cambia anche l'output cambia immediatamente.

Synchronous Sequential Logic

L'output corrente dipende dall'input corrente e anche dagli input passati, però l'output è non trasparente; quindi, se l'input cambia l'output non cambia immediatamente.

L'output cambia a seconda del valore del clock (*in comune a tutti i componenti di questo tipo*).

Macchina di Mealy

Classico esempio di logica asincrona. L'elemento di memoria è un D-Flip-Flop positive Edge triggered.

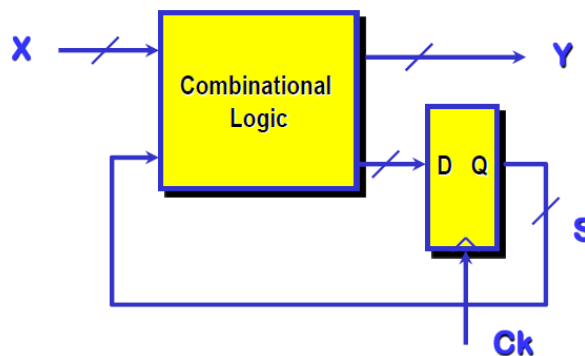


Figure 44 - Macchina di Mealy

Macchine di Mealy in cascata possono dare problemi a causa della loro trasparenza. Se l'input della prima balla, allora l'output della prima balla e così via.

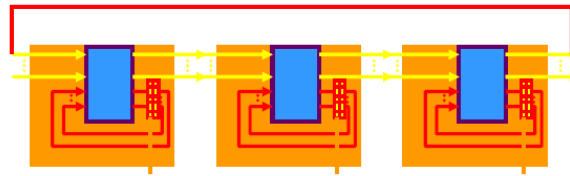


Figure 45- macchine di Mealy in cascata

Macchina di Moore

Classico esempio di logica sincrona. L'elemento di memoria è un D-Flip-Flop positive Edge triggered.

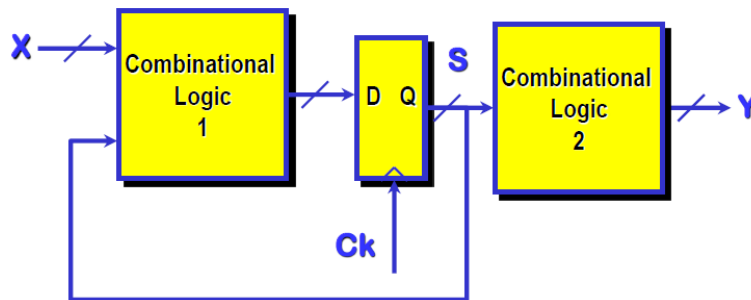


Figure 46 - Esempio di macchina di Moore.

Macchine di Moore in cascata non danno il problema delle macchine di Mealy.

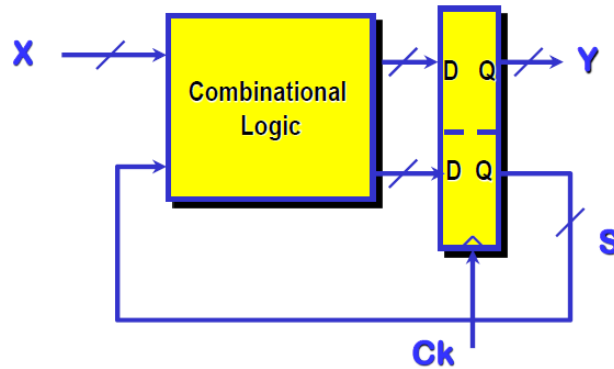


Figure 47 - Esempio di macchina di Moore.

Nodi

Hard Node

Se l'input non cambia, l'output non cambierà mai (es. Inverter).

Soft Node

Se l'input non cambiasse, l'output potrebbe comunque cambiare (es. memoria dinamica).

Questo accade quando utilizzo una cella di memoria dinamica: In questo tipo di cella, memorizziamo il bit in una capacità, eventualmente, la capacità si svuoterà e perderò le informazioni.

Questa è la situazione in cui memorizziamo il valore su una capacità parassita, un esempio potrebbe essere quando carichiamo un nodo di output attraverso un transistor di passaggio, la carica in questo caso si dissiperà lentamente a causa della corrente parassita, anche se manteniamo l'alimentazione accesa, non è garantito che il valore venga mantenuto, possiamo risolvere la situazione aggiornando il valore nella capacità.

Databook di un nodo

Informazioni date dal produttore. Come, ad esempio, lo schematico o la tabella di verità. Di solito contiene anche informazioni più particolari che andremo ad elencare.



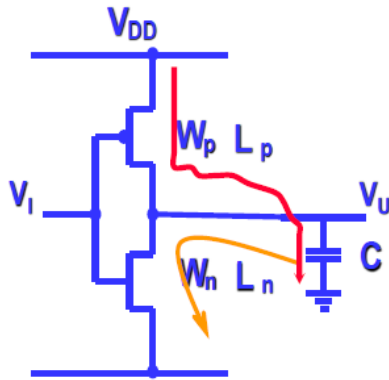
Figure 48- Schematico e tabella di verità

Propagation delay

Il propagation delay indica quanto tempo ci vuole affinché un cambiamento applicato al nodo di input si rifletta sul nodo di output del circuito. È un parametro critico nelle prestazioni di un circuito, specialmente in applicazioni ad alta velocità o ad alta frequenza, poiché influisce direttamente sulla velocità massima a cui il circuito può operare.

Questo ritardo è dovuto al tempo impiegato dal segnale per attraversare i vari componenti del circuito, come porte logiche, interconnessioni, buffer, ecc. Infatti, può essere influenzato da vari fattori, tra cui la complessità del circuito, la tecnologia di produzione, la lunghezza e la capacità delle interconnessioni, la velocità dei componenti logici utilizzati, la temperatura e altri fattori ambientali.

Misurare e minimizzare il propagation delay è importante per garantire prestazioni ottimali del circuito e soddisfare i requisiti di timing dell'applicazione. Questo può essere fatto tramite tecniche di progettazione appropriata, selezione accurata dei componenti e delle tecnologie utilizzate, e utilizzo di strumenti di simulazione per analizzare e ottimizzare il comportamento del circuito.



$$t_{pHL} \propto \frac{KC}{\beta_n} \frac{1}{V_{DD} - V_{Th}}$$

$$t_{pLH} \propto \frac{KC}{\beta_p} \frac{1}{V_{DD} + V_{Tp}}$$

$$\beta_n = \mu_n C_{ox} \frac{W_n}{L_n} \quad \beta_p = \mu_p C_{ox} \frac{W_p}{L_p}$$

Abbiamo delle formule in cui possiamo sfruttare tutti i possibili trade-off (Area, Consumo di Potenza, Soglia dei transistor, ecc.).

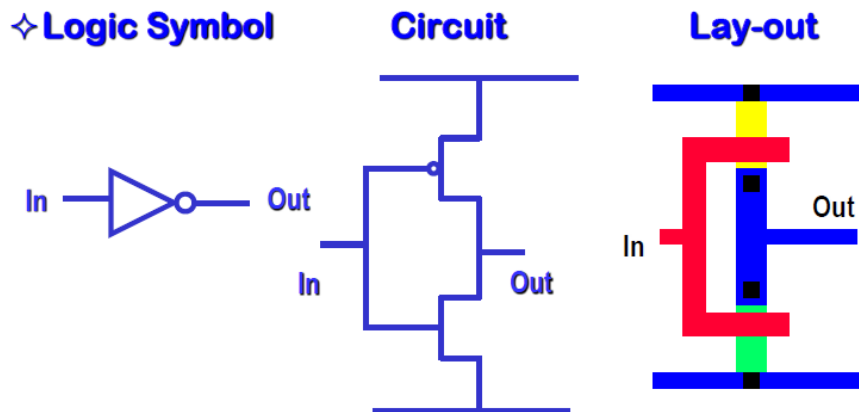
Alimentazione e ritardo di propagazione

Dalle formule possiamo vedere come **aumentando l'alimentazione, riduciamo il ritardo di propagazione.**

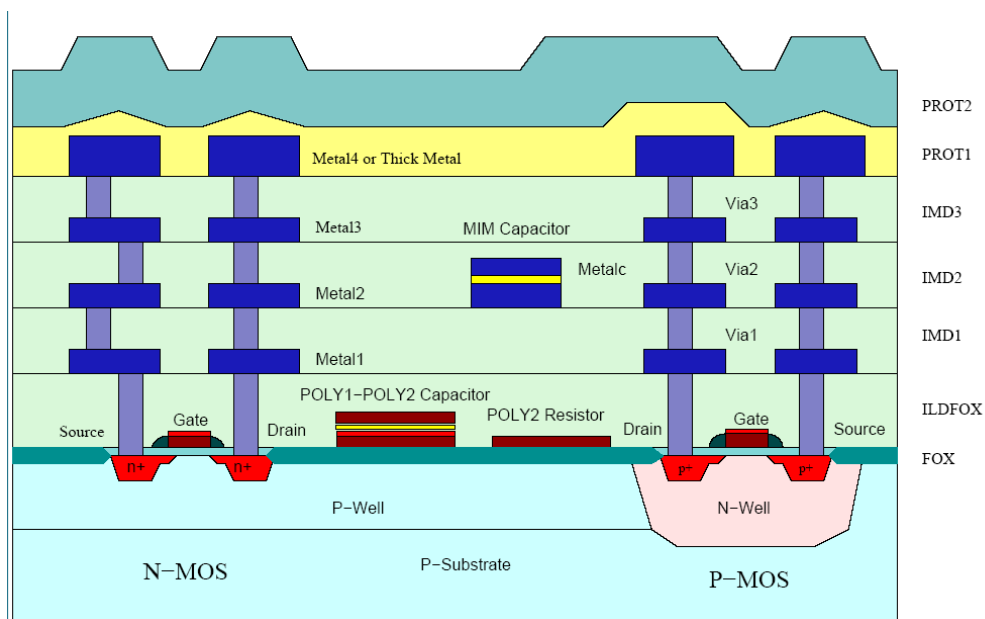
Soglia e ritardo di propagazione

Se riduciamo la soglia del transistor, diminuiamo il ritardo di propagazione, ma aumentiamo anche la corrente di dispersione e quindi il consumo energetico.

Interconnessione



Qui abbiamo i metalli (rettangoli blu) nelle interconnessioni, essi garantiscono maggiore flessibilità per le interconnessioni, ma sono necessarie le maschere che sono costose. Se riduciamo il numero di metalli, riduciamo anche il numero dei costi.



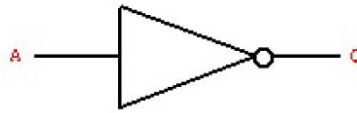
Strenght

Più è alta la forza, più è alta la corrente in uscita, e quindi possiamo caricare o scaricare più velocemente la capacità in uscita.

Quindi con maggiore forza ma con la stessa capacità di carico, otteniamo un ritardo di propagazione più basso, ma abbiamo una dimensione maggiore.

Un transistor più largo fornisce una corrente più alta e un minore ritardo di propagazione.

Strength	1
Cell Area	29.120 um ²
Equation	Q = "!A"
Type	Combinational
Input	A
Output	Q



State Table	
A	Q
L	H
H	L

Propagation Delay [ns]					
Input Transition [ns]		0.01		4.00	
Load Capacitance [fF]		5.00	100.00	5.00	100.00
A to Q	fall	0.04	0.41	-0.36	0.63
	rise	0.06	0.67	0.83	1.88

Output Transition [ns]					
Input Transition [ns]		0.01		4.00	
Load Capacitance [fF]		5.00	100.00	5.00	100.00
A to Q	fall	0.04	0.54	0.70	1.43
	rise	0.08	1.03	0.62	1.58

Leakage

La potenza perduta a causa di perdite.

Layout e cell area

Forma del circuito e area occupata dal layout del circuito, di solito espresso in picometri.

Wafer Cross-Section

Come i livelli sono posizionati sul wafer.

Technology Overview

Tabella che mostra i materiali e le tecniche usati per produrre il circuito.

Symbol of the node

Simbolo del nodo.

True Table

Esprime cosa fa il nodo.

Capacità

Capacità parassite di ingresso e uscita.

Strength

INVX1, il "1" rappresenta la forza del nodo. La forza è la capacità di pilotare il carico in uscita; quindi, alla quantità di corrente che il nodo è in grado di fornire in uscita. Una forza maggiore implica un minore ritardo di propagazione ma maggiore capacità e quindi maggiore consumo energetico.

Transition Time (Ritardo di propagazione)

Il tempo che il nodo impiega per cambiare il valore in uscita.

Il ritardo di propagazione dipende dalle condizioni operative (temperatura, alimentazione, processo, ecc.). Ma il valore in questa tabella è calcolato utilizzando determinati "technology corners".

Technology corners

Come posso stabilire le prestazioni di un nodo?

- **Fattori di produzione:** Alla fine del processo di produzione non tutti i nodi prodotti hanno le stesse caratteristiche, alcuni sono più veloci e altri sono più lenti.
- **Ritardo di propagazione:** Può dipendere da fattori esterni come la temperatura operativa (quindi la dalla mobilità degli elettroni), oppure l'alimentazione.

Quindi, i "technology corners" sono presenti perché come designer dobbiamo verificare il sistema al fine di tener conto di tali variazioni.

Propagation Delay [ns]				
Input Transition [ns]		0.01		4.00
Load Capacitance [fF]		5.00	100.00	5.00 100.00
A to Q	fall	0.04	0.41	-0.36 0.63
	rise	0.06	0.67	0.83 1.88

Output Transition [ns]				
Input Transition [ns]		0.01		4.00
Load Capacitance [fF]		5.00	100.00	5.00 100.00
A to Q	fall	0.04	0.54	0.70 1.43
	rise	0.08	1.03	0.62 1.58

Figure 49- Statistiche di prestazione di un dispositivo.

	Best	Typical	Worst
Temperatura	-40 - 0 C°	25 C°	85 - 125 C°
Alimentazione	1.1 * VDD	VDD	0.9 * VDD
Processo	FAST	TYPICAL	SLOW

Figure 50- Esempio di technology corners.

Typical

In questo caso quando abbiamo una condizione tipica, abbiamo valori di alimentazione e temperatura dell'ambiente tipici.

Worst

In questo caso se la condizione è peggiore, abbiamo:

- Un processo più lento.
- Una temperatura operativa molto alta.
- Alimentazione più bassa del 10%.

Best

In questo caso, se la condizione è migliore, abbiamo:

- Un processo più veloce.
- Una temperatura operativa molto bassa.
- Alimentazione più alta del 10%.

Ad alta velocità, V è più lento, soglia più bassa, bassa dispersione, ha una propagazione più lunga.

Maggiore capacità, maggiore consumo di energia e forse l'area con maggiore forza.

Quindi, per questo ritardo, nel Databook abbiamo i numeri per queste 3 condizioni (technology corners).

I ritardi di propagazione si dividono ancora in 2 numeri: H → L e L → H.

Le celle standard devono avere alta velocità, bassa dispersione e alta densità.

Rumore

Variazioni non volute del voltaggio e corrente nei nodi logici. Il rumore è dovuto agli elementi capacitivi (*rumore nel voltaggio*) e/o induttivi (*rumore nella corrente*) che si trovano tra due nodi.

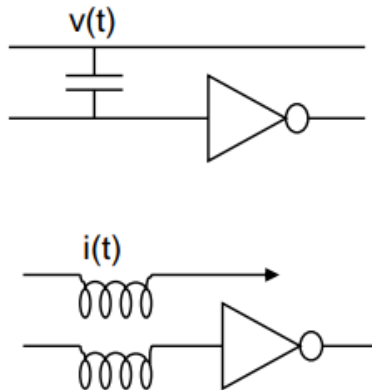


Figure 51- Tipiche situazioni in cui posso trovare rumore.

Il rumore può influenzare l'input di un nodo e creare comportamenti inaspettati.

Comportamento statico (Steady state parameters)

Dice quando il circuito è resistente alle variazioni dovute dal processo di manifattura e dal rumore. Una variabile logica è associata a due soglie di voltaggio.

Soglie

$$1 \Leftrightarrow V_{OH} \text{ and } 0 \Leftrightarrow V_{OL}$$

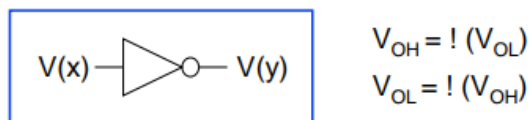


Figure 52- Soglie dei valori logici.

$$V_{sw} = V_{OH} - V_{OL}$$

Figure 53- Signal swing.

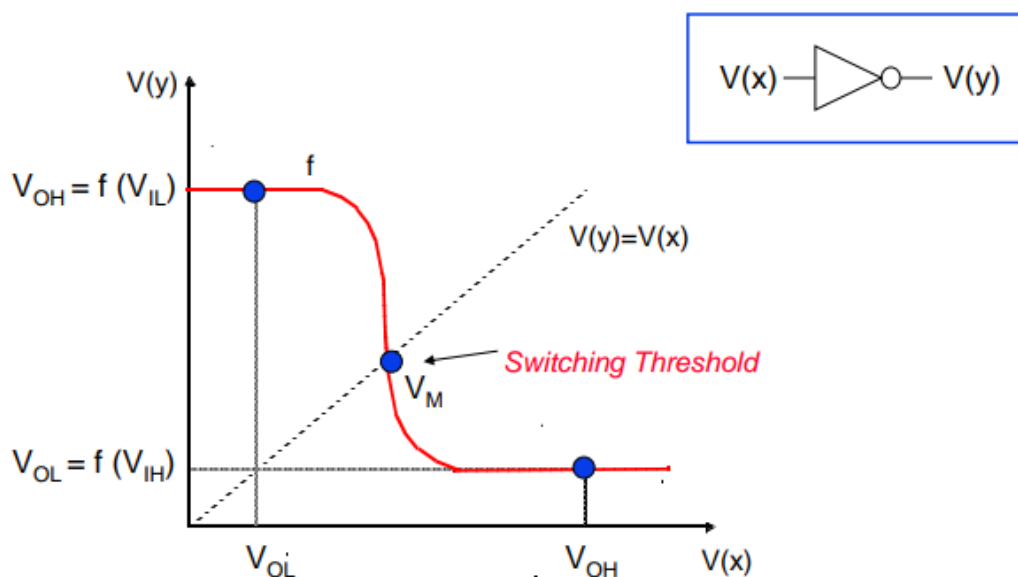


Figure 54- La posizione delle soglie determina la sensibilità dell'inverter, superata la soglia di switching il valore in uscita cambia.

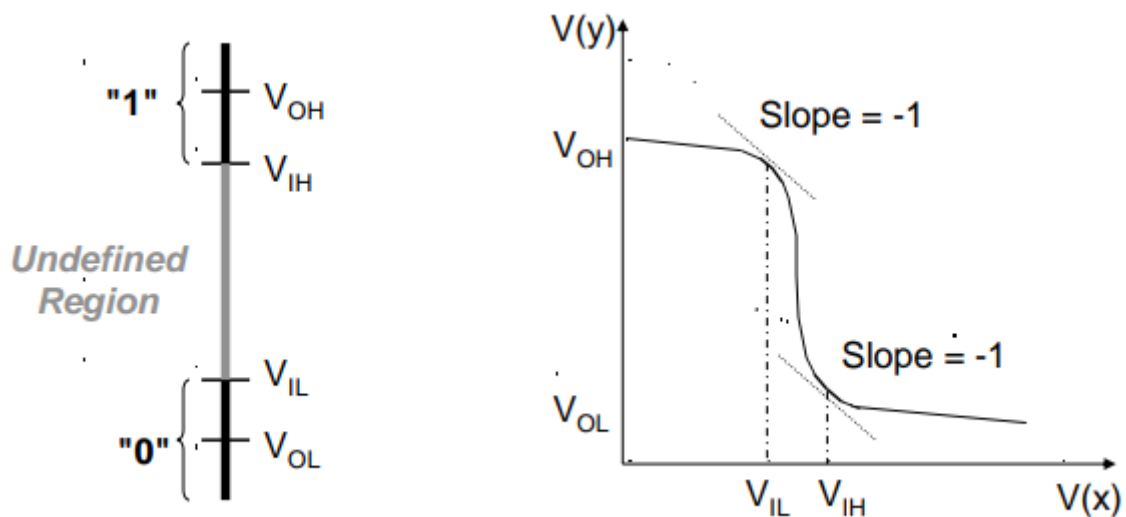


Figure 55- Le soglie non vengono scelte a caso.

La zona indefinita è il range di valori dell'input per cui ho un output non prevedibile.

Margini di rumore

Il nostro obiettivo è definire le soglie in modo che il nodo sia robusto a rumore in entrata. Quindi fare in modo che le zone definite (*quelle in nero*) siano più grandi possibile.

Transistors

CMOS Complementare

Con questa tecnica possiamo implementare qualsiasi funzione logica.

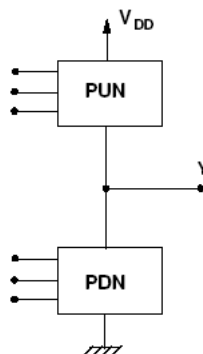


Figure 56- Schema di una rete implementata con complementary CMOS.

Dividiamo il nodo in due parti:

- **Pull-up network:** Connette l'output con l'alimentazione.
- **Pull-down network:** Connette l'output con la massa.

Pull-down Network (PDN)

Di solito la prima cosa che si fa è la PDN, che deriva direttamente dalla funzione logica da implementare.

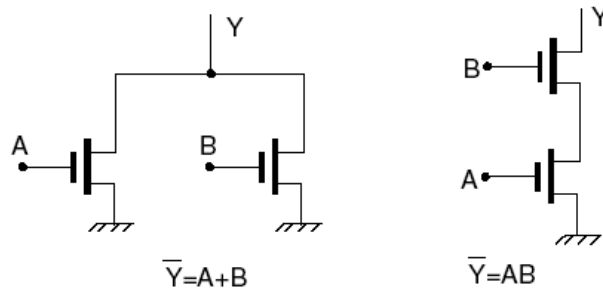


Figure 57- Esempio di pull-down network.

Nel primo caso (*quello a sinistra*), se $A=1$ OR $B=1$, colleghiamo l'uscita Y a massa. Se $A=1$, il transistor sinistro è approssimabile ad un cortocircuito, a cui colleghiamo la terra con Y (*la stessa cosa per B e il transistor destro*).

Pull-up Network (PUN)

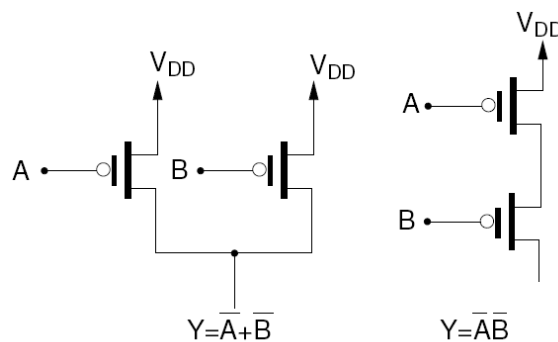


Figure 58- Esempio di pull-up network.

Nel primo caso (*a sinistra*), se $A=1$ OR $B=1$, colleghiamo l'uscita Y all'alimentazione.

$$\bar{Y} = f_{PDN}(A, B)$$

$$Y = f_{PUN}(\bar{A}, \bar{B})$$

Figure 59- La PDN e la PUN implementano due funzioni logiche diverse che se unite producono la funzione logica voluta.

NAND2

Implementiamo una NAND a due ingressi.

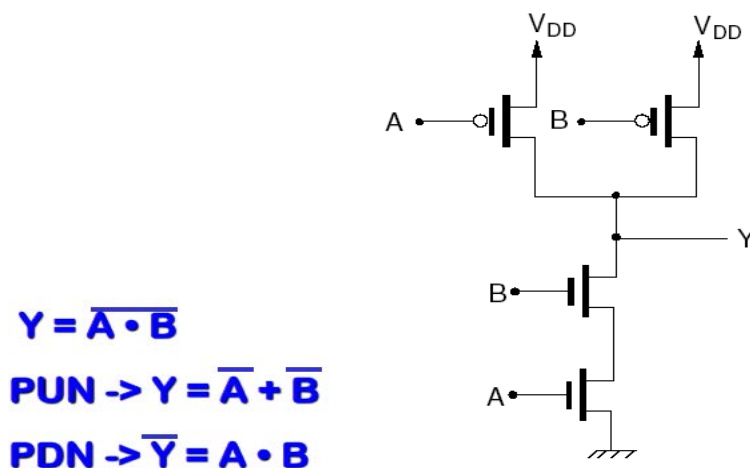


Figure 60- Schema della NAND.

NOR2

Implementiamo una NOR a due ingressi.

$$Y = \overline{A + B}$$

$$\text{PUN} \rightarrow Y = \overline{A} \cdot \overline{B}$$

$$\text{PDN} \rightarrow \overline{Y} = A + B$$

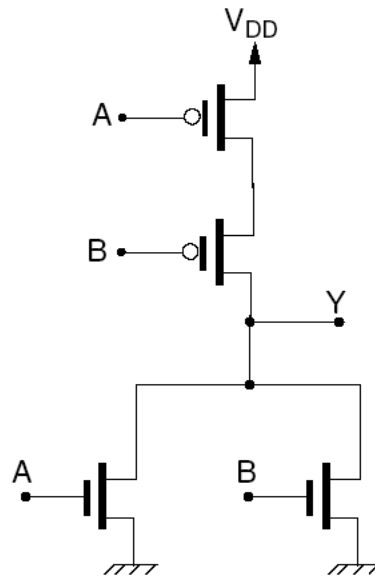


Figure 61- Schema della NOR.

Funzioni complesse (Teoria della dualità)

Una funzione complessa può essere prodotta con la legge della dualità.

$$Y = \overline{A(B + CD)}$$

$$\begin{aligned} \text{PUN} \rightarrow Y &= \overline{A(B + CD)} \\ &= \overline{A} + \overline{B + CD} \\ &= \overline{A} + \overline{B} \cdot \overline{CD} \\ &= \overline{A} + \overline{B}(\overline{C} + \overline{D}) \end{aligned}$$

$$\text{PDN} \rightarrow \overline{Y} = A(B + CD)$$

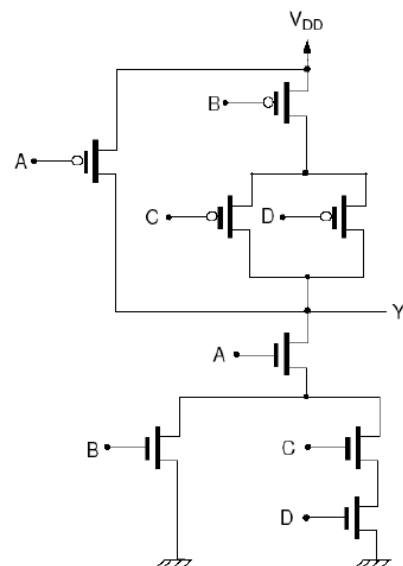


Figure 62- Esempio di utilizzo della teoria della dualità.

XOR

L'XOR non rispetta la teoria della dualità, quindi non può essere implementato con questa tecnica. Dobbiamo implementarlo manualmente. L'XOR non rispetta la teoria della dualità, quindi non può essere implementato con questa tecnica. Dobbiamo implementarla manualmente.

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

XOR

$$Y = A\bar{B} + \bar{A}B.$$

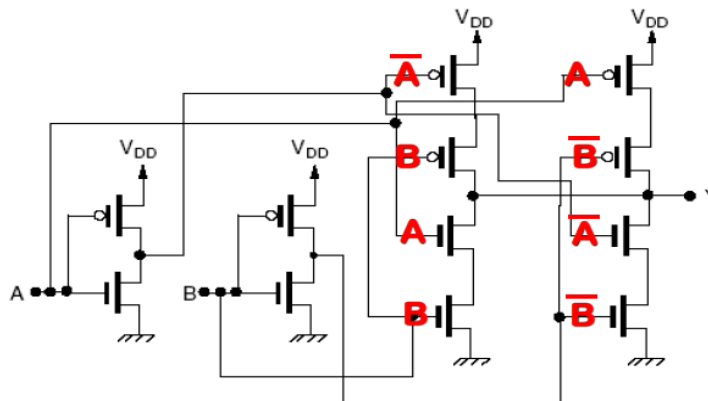


Figure 63- XOR implementata manualmente.

Complementary CMOS e il ritardo di propagazione

Dopo aver costruito una rete utilizzando questa tecnica come possiamo calcolare il ritardo di propagazione? Nel caso dell'inverter abbiamo queste due formule:

Dynamic A
$$t_{pHL} \propto \frac{KC}{\beta_n} \frac{1}{V_{DD} - V_{Tn}}$$

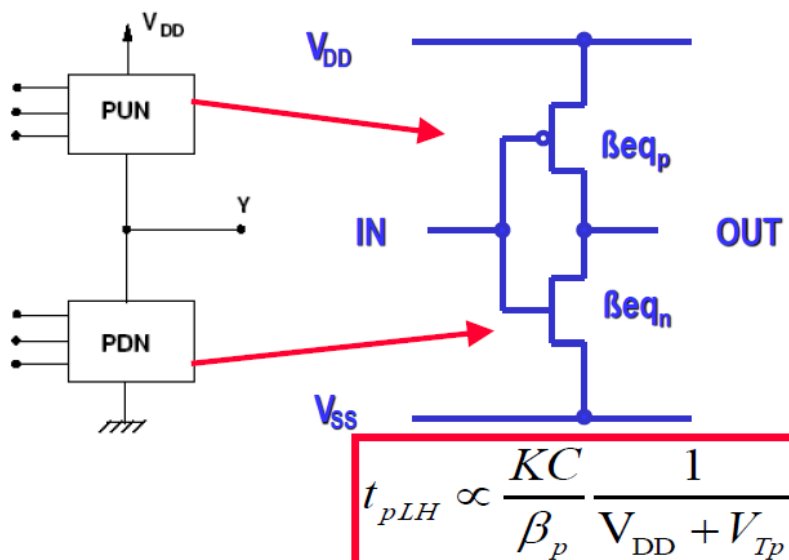


Figure 64- Formule dei due ritardi di propagazione.

$N < 4$

Transistor in parallelo

La conduttanza equivalente è la somma delle conduttanze.

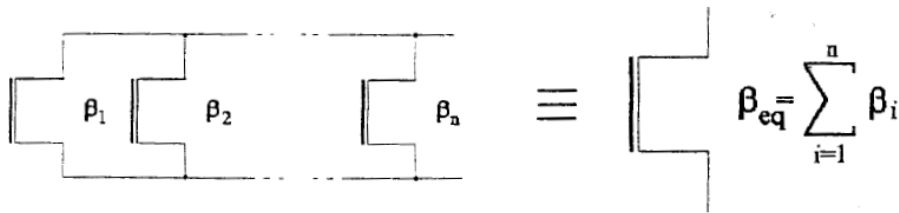


Figure 65- Transistor in parallelo.

Transistor in serie

La conduttanza equivalente è il reciproco della somma dei reciproci delle conduttanze.

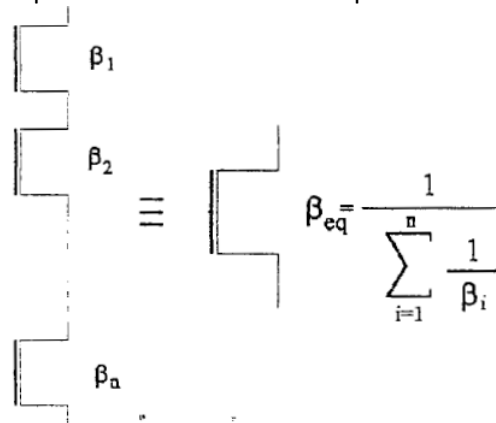
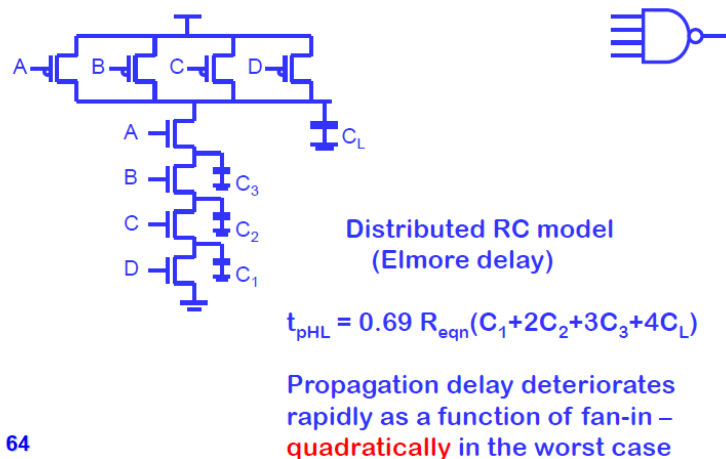


Figure 66- Transistor in serie.

Queste formule sono valide solo per un numero di transistor in parallelo/serie minore di 4, poiché oltre al 4 non posso più approssimare i transistor a componenti passivi.

$N \geq 4$

Se abbiamo 4 o più transistor, il ritardo aumenta in modo quadratico, poiché dobbiamo considerare anche la capacità (*non sono più approssimabili ad elementi passivi*):



64

Se ho M transistor in parallelo, considero 1 capacità.

Se ho N transistor in serie, considero $(N - 1)$ capacità.

Analisi Dinamica (NAND)

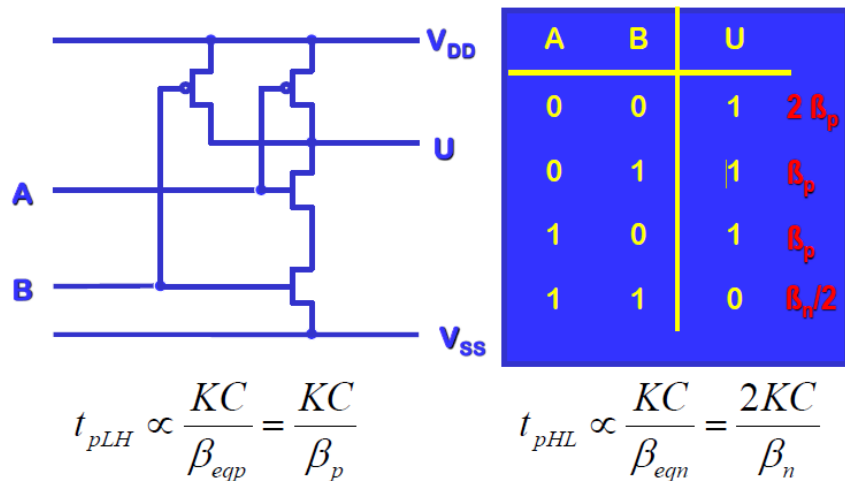


Figure 67- Analisi dinamica di una NAND.

Il mio obbiettivo è calcolare il beta_eq per ogni percorso.

Per il percorso $\{A, B\} = \{0, 0\}$ solo i PMOS saranno attivi.

I PMOS hanno $\beta = \beta_p$ e ho 2 PMOS in parallelo; quindi, il β_{eq} sarà la somma di tutti i β , che è $2 \cdot \beta_p$.

Per il percorso $\{A, B\} = \{0, 1\}$ solo un PMOS (A) e un NMOS (B) saranno attivi.

Il PMOS e il NMOS non sono collegati; quindi, il β_{eq} sarà uguale β_p , perché è il percorso tra Vdd e U.

Per il percorso $\{A, B\} = \{1, 0\}$ solo un PMOS (B) e un NMOS (A) saranno attivi.

Il PMOS e il NMOS sono collegati, ma il NMOS è in un circuito aperto quindi il GND non può raggiungere U e l'output sarà 1.

Il β_{eq} sarà uguale a β_p , perché è il percorso tra Vdd e U.

Per il percorso $\{A, B\} = \{1, 1\}$ solo gli NMOS saranno attivi.

Il NMOS ha $\beta = \beta_n$ e ho 2 NMOS in serie, quindi il β_{eq} sarà: il reciproco della somma dei reciproci dei β_n .

Consiglio per l'analisi dinamica

Gestendo l'area possiamo fare in modo che il ritardo $H \rightarrow L$ sia uguale a quello $L \rightarrow H$.

$$t_{pLH} \propto \frac{KC}{\beta_{eqp}} = \frac{KC}{\beta_p} \quad t_{pHL} \propto \frac{KC}{\beta_{eqn}} = \frac{2KC}{\beta_n}$$

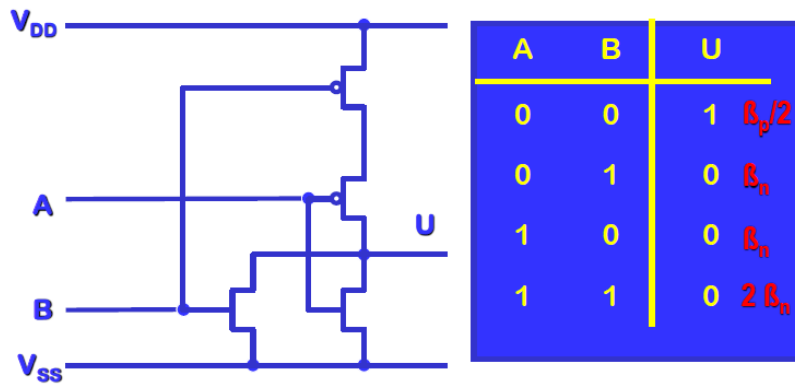
✧ **$t_{pHL} = t_{pLH}$ in the worst case**

$$\beta_p = \frac{\beta_n}{2} \rightarrow \beta_n = 2\beta_p \rightarrow \mu_n \frac{W_n}{L_n} = 2\mu_p \frac{W_p}{L_p}$$

If $\mu_p = \frac{\mu_n}{2} \quad L_n = L_p = L_{\min} \quad \rightarrow \quad W_n = W_p$

Figure 68- Modificando l'area del PMOS e/o NMOS si può fare in modo che il rapporto nella seconda riga sia lo stesso.

Analisi dinamica di una NOR



52 If $\mu_p = \frac{\mu_n}{2} \quad L_n = L_p = L_{\min} \quad \rightarrow \quad W_p = 4W_n$

Figure 69- Analisi dinamica di una NOR

Confrontare due soluzioni: NOR vs NAND

Applichiamo il consiglio di prima, sia nella NOR che nella NAND. Quindi il ritardo di propagazione $H \rightarrow L$ è uguale a quello $L \rightarrow H$.

$$\beta_{eqNAND} = \beta_p^{NAND} = \frac{\beta_n^{NAND}}{2}$$

$$\beta_{eqNOR} = \beta_n^{NOR} = \frac{\beta_p^{NOR}}{2}$$

$$W_n^{NOR} = W \quad W_p^{NOR} = 4W$$

$$\beta_n^{NAND} = 2\beta_n^{NOR} \Rightarrow W_n^{NAND} = 2W_n^{NOR} = 2W$$

$$\beta_p^{NAND} = \frac{\beta_p^{NOR}}{2} \Rightarrow W_p^{NAND} = \frac{W_p^{NOR}}{2} = \frac{4W}{2} = 2W$$

Per avere la simmetria dei due ritardi di propagazione in una porta NAND, dobbiamo garantire che le condizioni di ingresso che attivano ciascun percorso attraverso la porta siano equivalenti. In altre parole, sia il percorso attraverso l'ingresso PMOS che quello attraverso l'ingresso NMOS devono avere la stessa complessità e/o lo stesso numero di transistor attivi.

Ciò significa che dovremmo avere lo stesso numero di transistor PMOS e NMOS attivi in ciascun percorso o comunque che la complessità dei percorsi sia sempre la stessa. Questo assicurerà che i ritardi di propagazione siano simmetrici.

Quando entrambe le porte, NAND e NOR, hanno la stessa velocità, ciò significa che i loro ritardi di propagazione sono uguali o simili. Questo implica che il ritardo attraverso i percorsi critici di entrambe le porte è bilanciato, il che può avvenire regolando le dimensioni dei transistor e altri parametri del circuito.

At the end of the story

NOR $\rightarrow 4W + W \rightarrow (x2) \rightarrow 10W$

NAND $\rightarrow 2W + 2W \rightarrow (x2) \rightarrow 8W$

La porta NOR è più complessa della NAND.

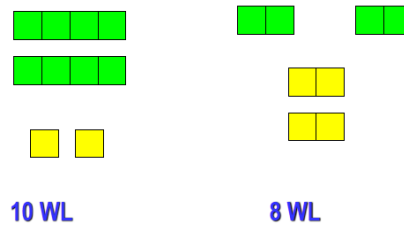


Figure 70 - A sinistra la NOR e a destra la NAND.

Percorso critici di una rete combinatoria

Colorati in rosso e verde abbiamo i 2 percorsi più critici, ossia quelli che attraversano più Transistors.

A	B	C	D	Y	B
0	0	0	0	1	5/3
0	0	0	1	1	3/2
0	0	1	0	1	3/2
0	0	1	1	1	1
0	1	0	0	1	1
0	1	0	1	1	1
0	1	1	0	1	1
0	1	1	1	1	1
1	0	0	0	1	2/3
1	0	0	1	1	1/2
1	0	1	0	1	1/2
1	0	1	1	0	1/3
1	1	0	0	0	1
1	1	0	1	0	1
1	1	1	0	0	1
1	1	1	1	0	4/3

$$Y = \overline{A(B + CD)}$$

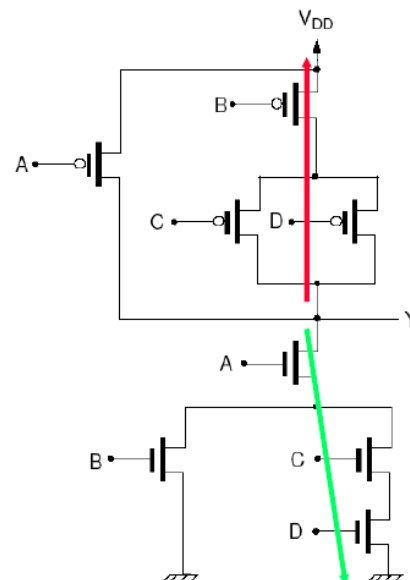


Figure 71- I percorsi critici.

Vediamo come aggiustare le cose:

$$t_{pLH} \propto \frac{KC}{\beta_{eq_p}} \qquad t_{pHL} \propto \frac{KC}{\beta_{eq_n}}$$

$$\beta_{eq_p} = \frac{\beta_p}{2} = \frac{\mu_p}{2} \frac{W_p}{L_p} \qquad \beta_{eq_n} = \frac{\beta_n}{3} = \frac{\mu_n}{3} \frac{W_n}{L_n}$$

✧ $t_{pHL} = t_{pLH}$ in the worst case

$$\frac{\mu_p}{2} \frac{W_p}{L_p} = \frac{\mu_n}{3} \frac{W_n}{L_n}$$

If $\mu_p = \frac{\mu_n}{2} \quad L_n = L_p = L_{\min} \quad \longrightarrow \quad \frac{W_p}{W_n} = \frac{4}{3}$

Figure 72- Calcoli

Analizziamo i percorsi critici:

In quello dei PMOS abbiamo un PMOS in serie a 2 PMOS in parallelo, quindi β_{eq} è $\beta_p/2$.

In quello dei NMOS abbiamo 3 NMOS in serie, quindi β_{eq} è $\beta_n/3$.

I due β_{eq} devono essere uguali, quindi per renderli tali operiamo in W_p e W_n .

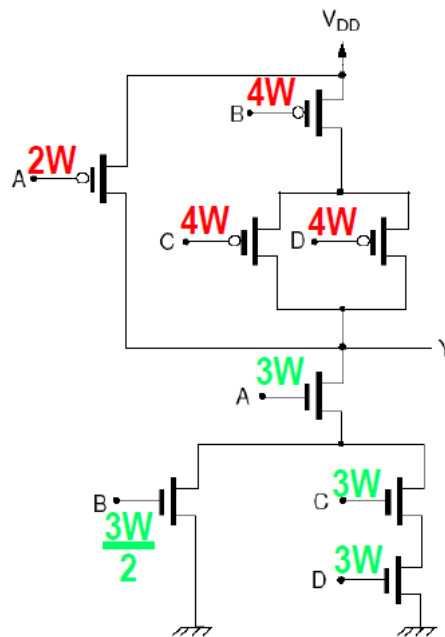


Figure 73- Soluzione ai percorsi critici.

Aggiustando l'area di ogni transistor posso fare in modo che ogni percorso abbia la stessa complessità e che quindi non ne esista uno critico.

CMOS gate – Body effect

Il Body Effect nei circuiti CMOS è un fenomeno che si verifica a causa della differenza di potenziale tra il substrato di silicio (body) e la sorgente del transistor MOSFET.

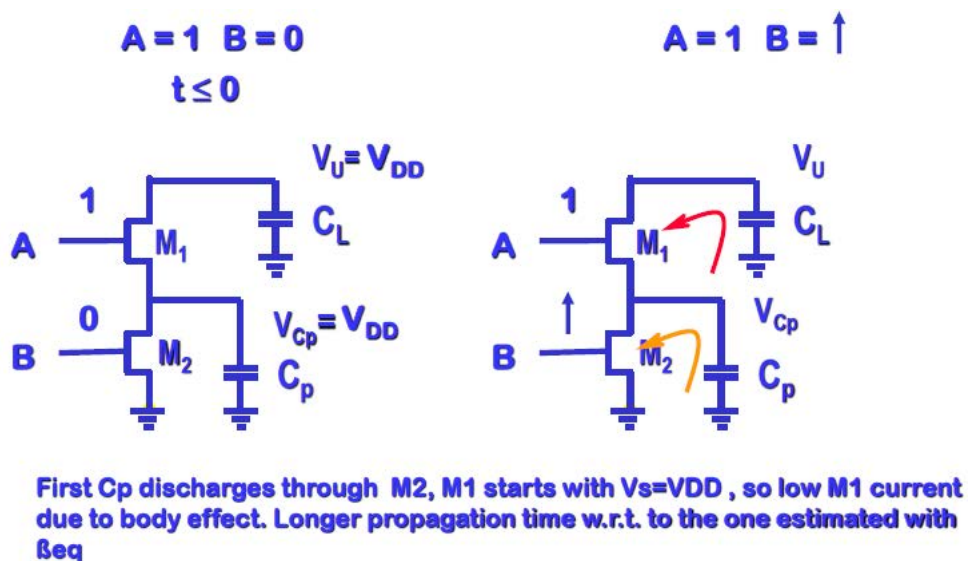
Nei transistor MOSFET, la regione di silicio sotto il canale è di solito connessa al substrato, che può essere portato a un potenziale diverso rispetto alla sorgente del transistor. Questa differenza di potenziale può influenzare le caratteristiche elettriche del transistor.

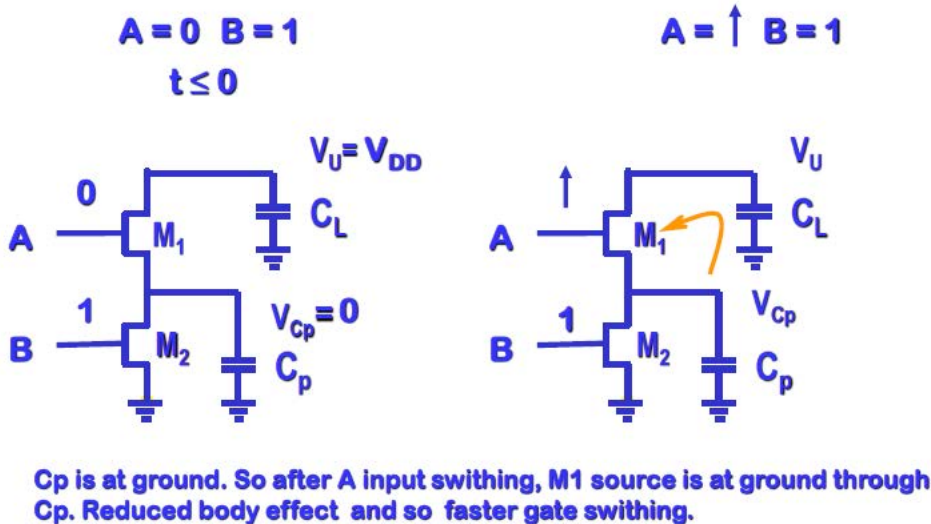
Nel caso dei transistor CMOS, che utilizzano sia transistor N-channel (NMOS) che transistor P-channel (PMOS) all'interno della stessa cella logica, l'effetto di corpo può influenzare in modo significativo le prestazioni del circuito.

L'effetto di corpo è particolarmente rilevante nei transistor P-channel, dove il substrato di silicio è generalmente collegato al potenziale di alimentazione positivo (Vdd). Quando il transistor P-channel è acceso, il substrato assume il potenziale di Vdd, il che crea una differenza di tensione tra il substrato e la sorgente del transistor. Questa differenza di tensione riduce la larghezza del canale effettivo del transistor, aumentando così la resistenza del canale e riducendo la conducibilità del transistor stesso. Ciò porta a una riduzione della corrente di drain (ID) del transistor P-channel, influenzando negativamente le prestazioni del circuito.

Per mitigare l'effetto di corpo, possono essere adottate diverse strategie, tra cui l'uso di strutture di transistor speciali, la modifica delle tensioni di polarizzazione o l'utilizzo di tecniche di progettazione specifiche che tengono conto di questo fenomeno.

È importante considerare l'effetto di corpo durante la progettazione di circuiti CMOS per garantire che le prestazioni del circuito siano robuste e coerenti, specialmente nelle applicazioni ad alta precisione o a bassa potenza.





Le conseguenze dell'effetto di corpo (Body Effect) nei transistor CMOS possono essere diverse e includono:

Variazione delle caratteristiche di guadagno e corrente: L'effetto di corpo può influenzare le caratteristiche di guadagno e corrente dei transistor CMOS, specialmente nei transistor P-channel. Questo può portare a una variazione delle prestazioni del circuito, come una maggiore resistenza di canale e una minore corrente di drain del transistor P-channel, influenzando il funzionamento del circuito.

Variazione del ritardo di propagazione: L'effetto di corpo può influenzare il ritardo di propagazione dei segnali attraverso i transistor CMOS, poiché **può influenzare la velocità di commutazione dei transistor**. Ciò può portare a una variazione del ritardo di propagazione tra i percorsi critici del circuito, influenzando il tempo di risposta complessivo del circuito.

Effetto sulla tensione di soglia: L'effetto di corpo può influenzare la tensione di soglia dei transistor CMOS, specialmente nei transistor P-channel. Questo **può portare a una variazione della tensione di soglia dei transistor e quindi delle loro caratteristiche operative**, influenzando le prestazioni del circuito.

Consumo di potenza: L'effetto di corpo può influenzare il consumo di potenza dei transistor CMOS, specialmente nei transistor P-channel, poiché può portare a una riduzione della corrente di drain e quindi a una maggiore resistenza di canale. Questo può aumentare la potenza dissipata nei transistor e ridurre l'efficienza energetica complessiva del circuito.

Fan-in

Indica il numero di input (ingressi) che possono essere collegati a una porta logica senza comprometterne il funzionamento corretto. In altre parole, il numero massimo di segnali di input che può gestire contemporaneamente.

Influisce sulla complessità del circuito, sulle prestazioni e sulla velocità di commutazione. Idealmente, **si desidera mantenere il fan-in il più basso possibile** per evitare ritardi eccessivi e mantenere la stabilità del circuito.

Ad esempio: in una porta logica AND con un fan-in di 2, significa che la porta può accettare fino a due segnali di input. Se si tenta di collegare più di due segnali di input, la porta potrebbe non funzionare correttamente o potrebbe verificarsi un ritardo maggiore nella propagazione del segnale attraverso la porta.

Consiglio sul fan-in

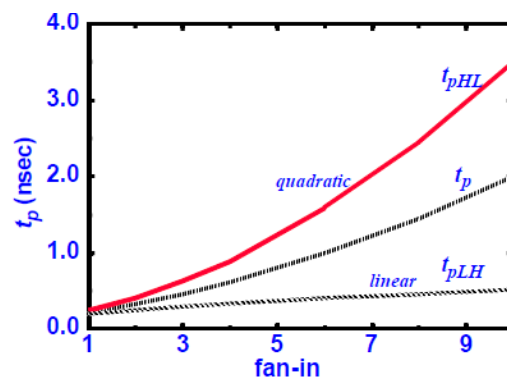


Figure 74- Effetti del fan-in sul ritardo di propagazione.

Porte con un fan-in maggiore di 4 dovrebbero essere evitate, perché può portare ad un aumento esponenziale del ritardo di propagazione.

Esempio di analisi dinamica (NAND)

$$Y = \overline{A_1 \cdot A_2 + B_1 \cdot B_2}$$

PDN $\overline{Y} = A_1 \cdot A_2 + B_1 \cdot B_2$

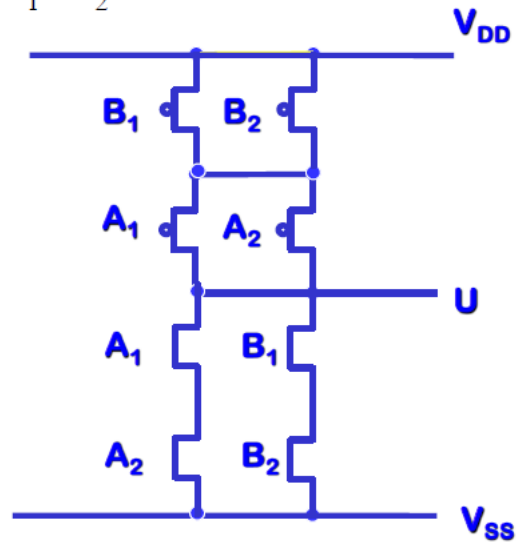


Figure 75 - Schematico della NAND.

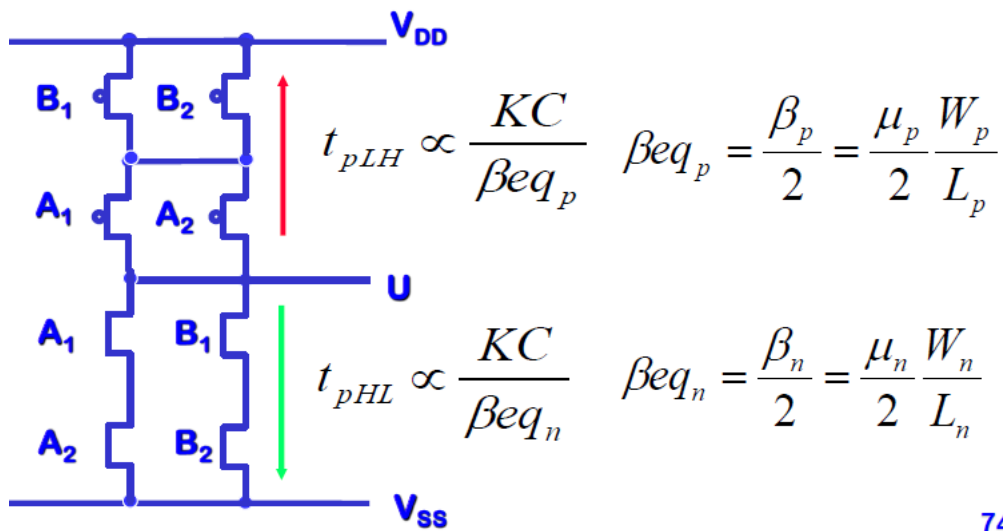


Figure 76 - Calcoli.

74

Catena di inverter

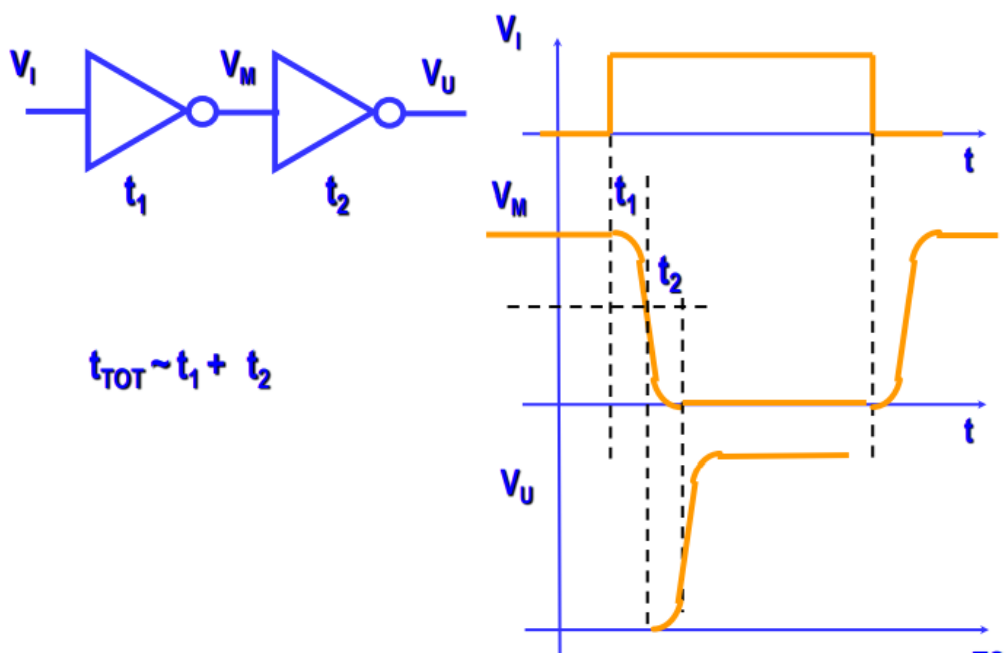


Figure 77- Una catena composta da due inverter e il comportamento equivalente risultante.

Timing

Il timing è molto importante quando abbiamo sistemi con registri. Per timing si intende il come regolare il clock, il quale è in comune per ciascun registro.

Analizzando i percorsi RLR (*registro-logica-registro*) possiamo verificare facilmente i requisiti di timing del sistema.

Latch

I latch sono sensibili al valore del clock.

- Se il clock è 1, output trasparente all'input.
- Se il clock è 0, output non è trasparente all'input e mostra l'ultimo valore dell'input (*memorizzazione*).

Flip Flop Edge Triggered (Registro)

I flip-flop sono sensibili a cambio di valore del clock, l'output cambia solo al fronte positivo del clock (quando diventa 1).

- Quando il clock è 1 e c'è una transizione, l'uscita assume il valore dell'input corrente.
o Durante questo tempo, il registro è trasparente.
- Quando il clock è 0, l'input non viene più ascoltato e l'uscita è conservata.

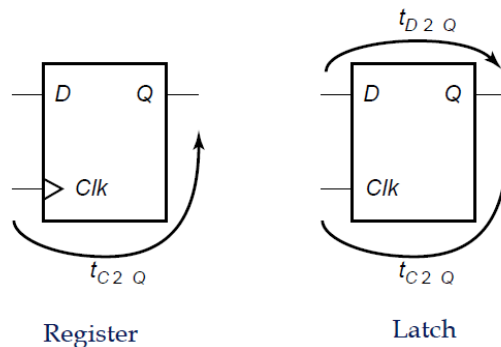


Figure 78 - Un registro e un latch a confronto.

Clock Skew

La conseguenza di usare i registri è che tutti i circuiti che li includono dipendono dal clock.

I registri devono vedere tutti lo stesso fronte di salita del clock.

Questa condizione viene meno a fronte di in una grande area; in questo caso c'è uno sfasamento del clock, che è un delta (ritardo) tra il timing del clock di due registri posizionati in due punti distanti del circuito.

Anche un piccolo ritardo può causare problemi di desincronizzazione, i quali producono effetti imprevedibili.

Gals (Global Asynchronous Locally Synchronous)

At the end of the story: avere un sistema con un design enorme possiamo utilizzare GALS.

Dividiamo il nostro chip in diverse aree locali, ognuna delle quali ha il suo clock locale.

Ogni area è collegata con altre aree attraverso una comunicazione asincrona.

Questo approccio serve a ridurre lo sfasamento del clock. Il nuovo problema è quello di gestire la comunicazione tra le aree, che funziona utilizzando diverse frequenze di clock.

Timing classification

Supponiamo che dobbiamo impostare il timing tra due registri per un'operazione di registro-registro. Dobbiamo comprendere i vincoli temporali:

Tsetup e Thold

L'input deve essere stabile per un tempo TSetup prima del fronte di salita del clock e per un tempo THold dopo il fronte di salita del clock.

Tc-q

Il tempo tra il fronte di salita del clock e l'istante in cui l'uscita diventa stabile.

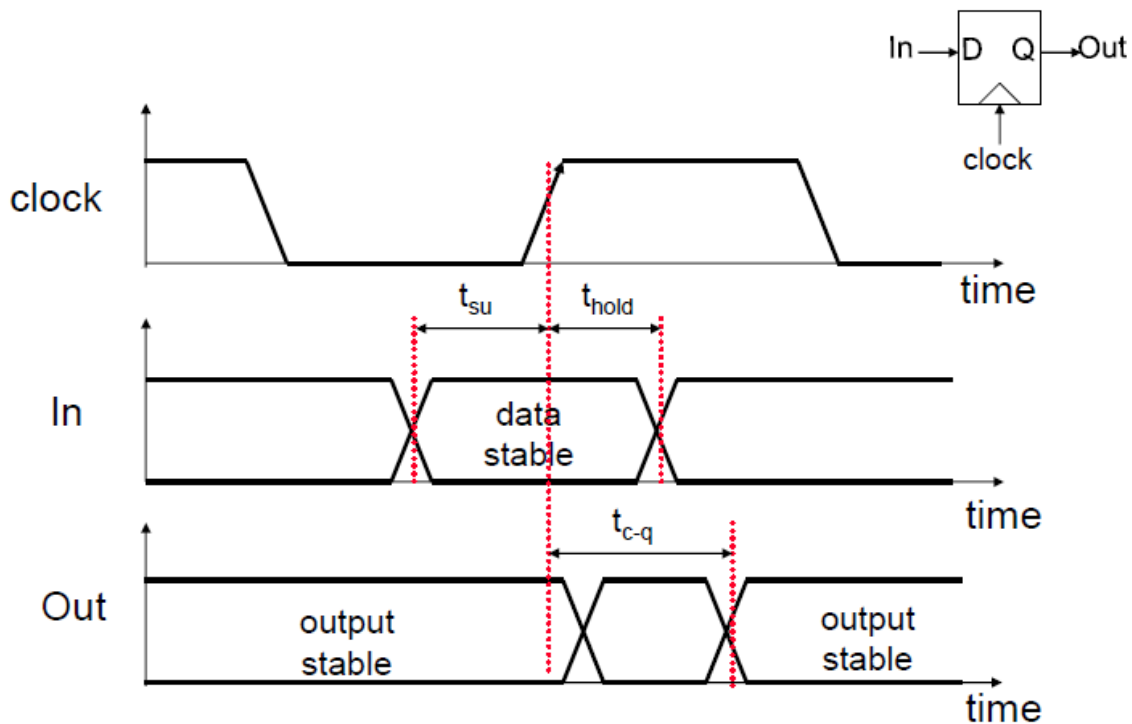


Figure 79- TSetup, THold e Tc-q in azione.

Tplogic

Una rete logica possiede un tempo di attraversamento (*tempo di propagazione*) che va contato e che di sicuro è maggiore rispetto al tempo di propagazione su un semplice filo.

Worst possible propagation delays

Definiamo anche il ritardo di propagazione tPHL e tPLH, che sono i peggiori ritardi di propagazione possibili.

Tpcdlogic

In una rete logica posso avere glitch o comunque instabilità temporanee sull'output. Definiamo il ritardo di contaminazione (tcdlogic) come il ritardo dalla transizione dell'input al PRIMO cambiamento dell'output. Nota che il primo cambiamento non è quello "buono" ma è quando l'output viene contaminato a causa del cambiamento dell'input, non è quando l'output diventa stabile.

Timing violations

Supponiamo di avere due registri R1 e R2, collegati da una rete combinatoria.

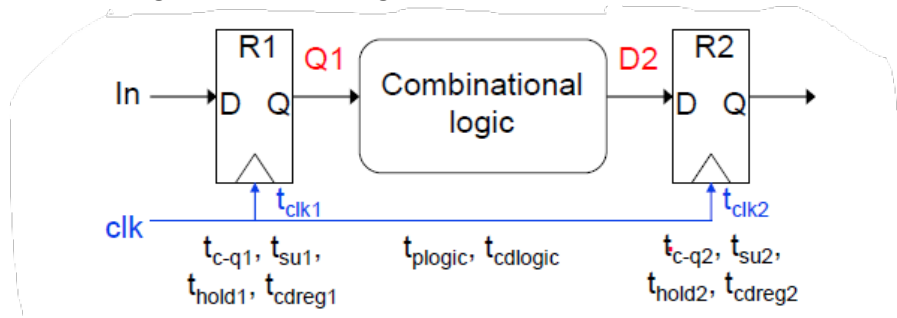


Figure 80- Rete di esempio.

Supposizioni

- R1 e R2 hanno lo stesso clock con periodo T.
- Non ci importa (e non sappiamo nulla) di cosa succede prima di R1.
- La logica combinatoria introduce un ritardo T_{plogic}.

Supponiamo che al fronte di salita del clock l'input di R1 cambi.

- 1 Prima del fronte di salita del clock, IN deve essere stabile per T_{setup1}.
 - 2 Fronte di salita del clock → R1 e R2 sono in ascolto degli ingressi.
 - 3 Dopo T_{cq1} → Q1 cambia e diventa stabile.
 - 4 D2 (input di R2) deve essere stabile per almeno T_{hold2}.
 - 5 Dopo T_{cq2} → Q2 cambia e diventa stabile.
 - 6 Fronte di discesa del clock → R1 e R2 non sono più in ascolto degli ingressi.
- Dobbiamo fare in modo che T sia abbastanza grande da soddisfare questi due requisiti.

Setup time violation

$$T \geq t_{cq1} + t_{plogic} + t_{setup2}$$

Quando si viola la prima regola: : D2 cambia subito prima del fronte di salita.

- T_{cq1} → Tempo richiesto da R1 per adeguare l'uscita all'ingresso.
- T_{plogic} → Tempo richiesto per attraversare la rete combinatoria.
- T_{setup2} → Tempo richiesto da R2 per campionare in modo corretto.

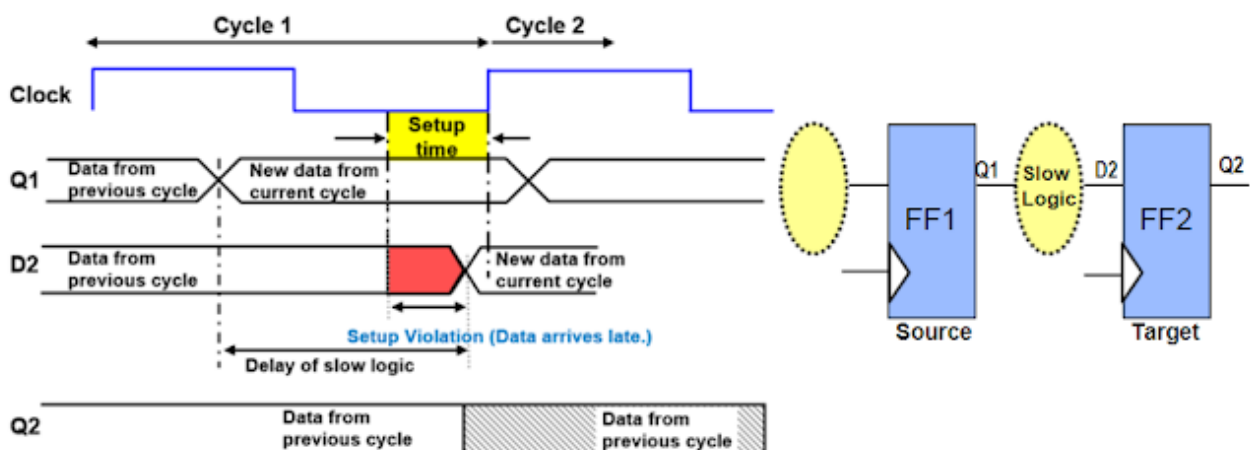


Figure 81- Esempio di setup violation, in questo caso causata da una rete combinatoria molto lenta.

Hold time violation

$$t_{\text{hold}2} \leq t_{\text{cdReg1}} + t_{\text{cdLogic}}$$

Quando si viola la seconda regola: **D2 cambia subito dopo al fronte di salita.**

- t_{cdReg1} → Tempo richiesto a D1 per propagarsi attraverso R1 e uscire come segnale di uscita stabile (ossia quanto tempo serve al registro per aggiornare l'uscita).
- t_{cdLogic} → Tempo di contaminazione della rete combinatoria, ossia il tempo che trascorre dalla variazione dell'input alla stabilizzazione dell'output della rete combinatoria (ossia quanto è reattiva la rete).

Se la condizione è rispettata, allora (supponendo che il segnale D1 cambi contemporaneamente al fronte di salita del clock) la propagazione del nuovo segnale sarà troppo lenta per influire sul campionamento del segnale precedente e quindi non avrà conseguenze indesiderate.

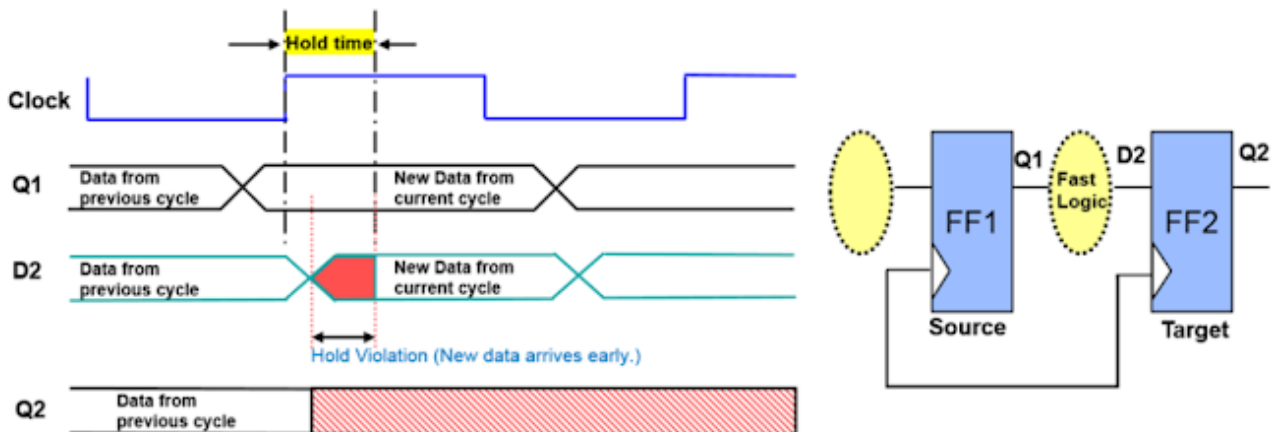


Figure 82- Hold violation, causato da una rete combinatoria troppo veloce.

Come evitarle?

Per evitare queste due problematiche è possibile impostare bene il periodo di clock oppure utilizzando tecniche di sincronizzazione, come il ritardo dei segnali di dati rispetto al segnale di clock (*clock queue*), per garantire il corretto funzionamento del registro e prevenire violation.

Clock Queue

Timing e strumenti di sintetizzazione

1. Lo strumento trova ogni possibile percorso che un segnale potrebbe seguire.
2. Per ogni percorso, calcolerà il tempo di propagazione lungo quel percorso.
3. **Il percorso critico è quello con il maggiore tempo di propagazione**, che determina anche la durata minima del periodo di clock.

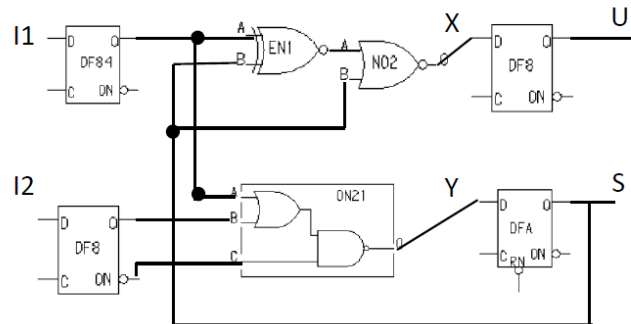


Figure 83 - Rete di esempio.

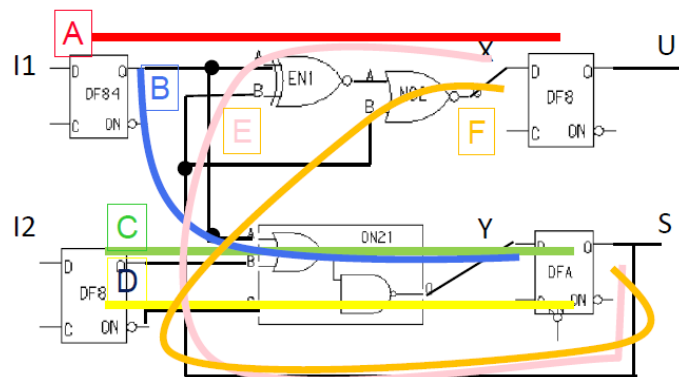
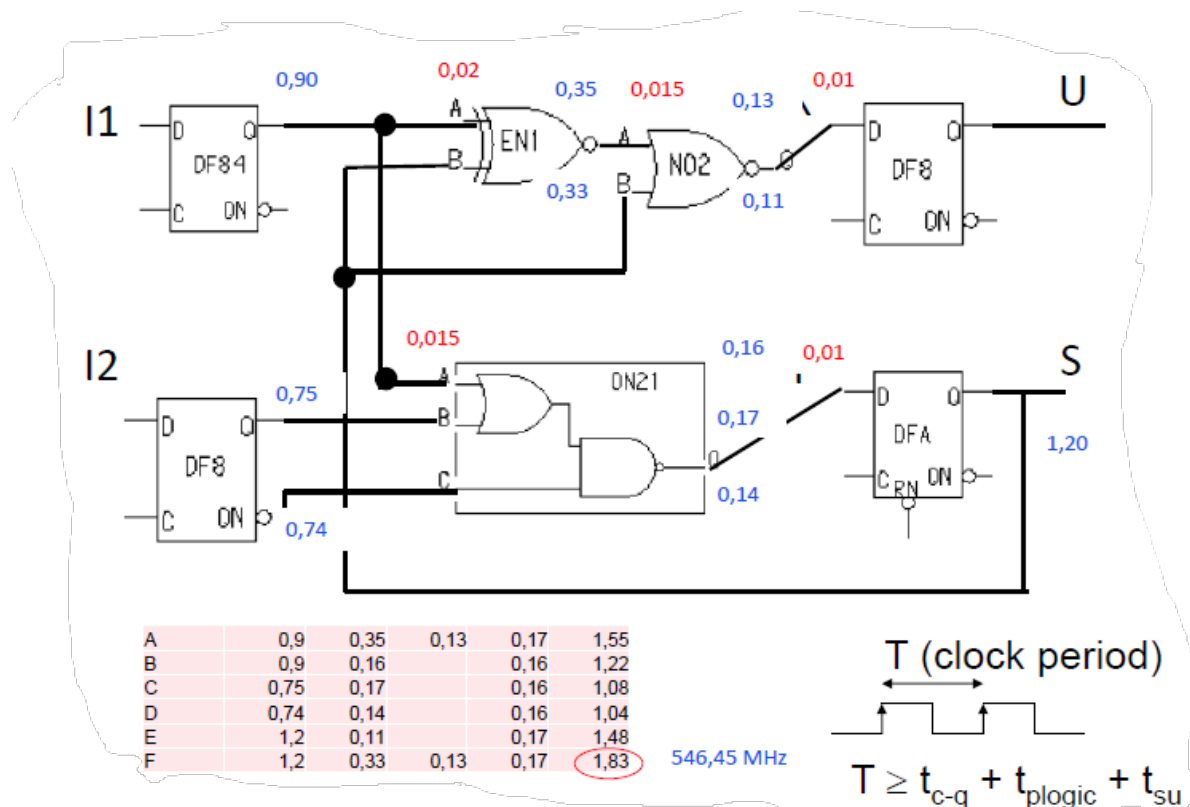


Figure 84 - Calcolo di ogni possibile percorso RLR (Register-Logic-Register).

Per ogni componente, il tool possiede il databook dal quale legge ogni possibile informazione utile:

- Tabella di verità → Per escludere percorsi che non avverranno mai.
- Area.
- Potenza dissipata.
- Tempo di propagazione per ogni ingresso per ogni uscita.
- Capacità di **ingresso** e di **uscita**.
- Nel caso dei registri: Tsetup e Thold.



In questo caso lo strumento ci dice che il percorso F è quello che richiede più tempo, quindi lui è quello critico e il periodo clock deve essere maggiore di quel tempo di propagazione.

Sources of clock Skew and Jitter in a clock network

$$T \geq t_{c-q} + t_{logic} + t_{su}$$

$$t_{hold} \leq t_{cdlogic} + t_{cdreg}$$

Figure 85 - Le due regole.

Supponiamo che il clock arrivi allo stesso istante presso R1 e R2.

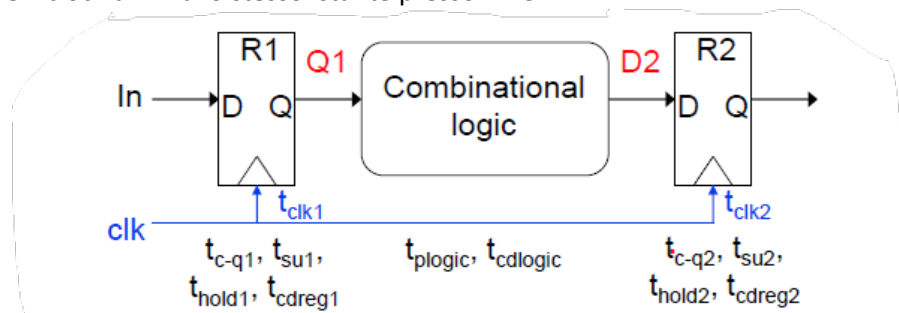


Figure 86- Rete di esempio

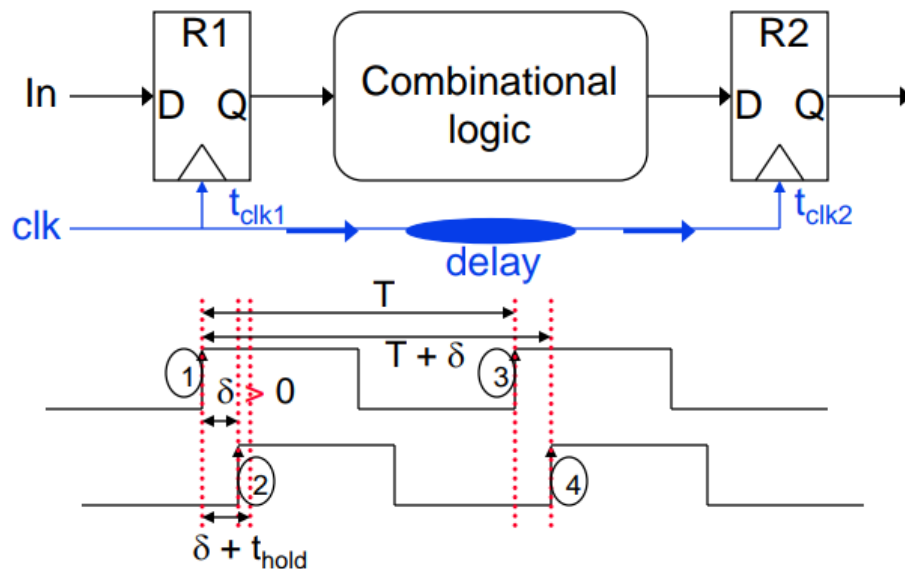
Se vogliamo analizzare queste equazioni, dobbiamo considerare lo scostamento (Skew) di clock e il jitter del clock.

Clock skew

Parametro statico dovuto alle lunghezze diverse dell'interconnessione (*dei fili*), carichi diversi e da agenti esterni come la temperatura o l'alimentazione.

Questo fenomeno non sempre è negativo, a volte può giocare a nostro favore per far rispettare i requisiti di timing.

Positive clock skew



$$T: \quad T + \delta \geq t_{c-q} + t_{plogic} + t_{su} \quad \text{so} \quad T \geq t_{c-q} + t_{plogic} + t_{su} - \delta$$

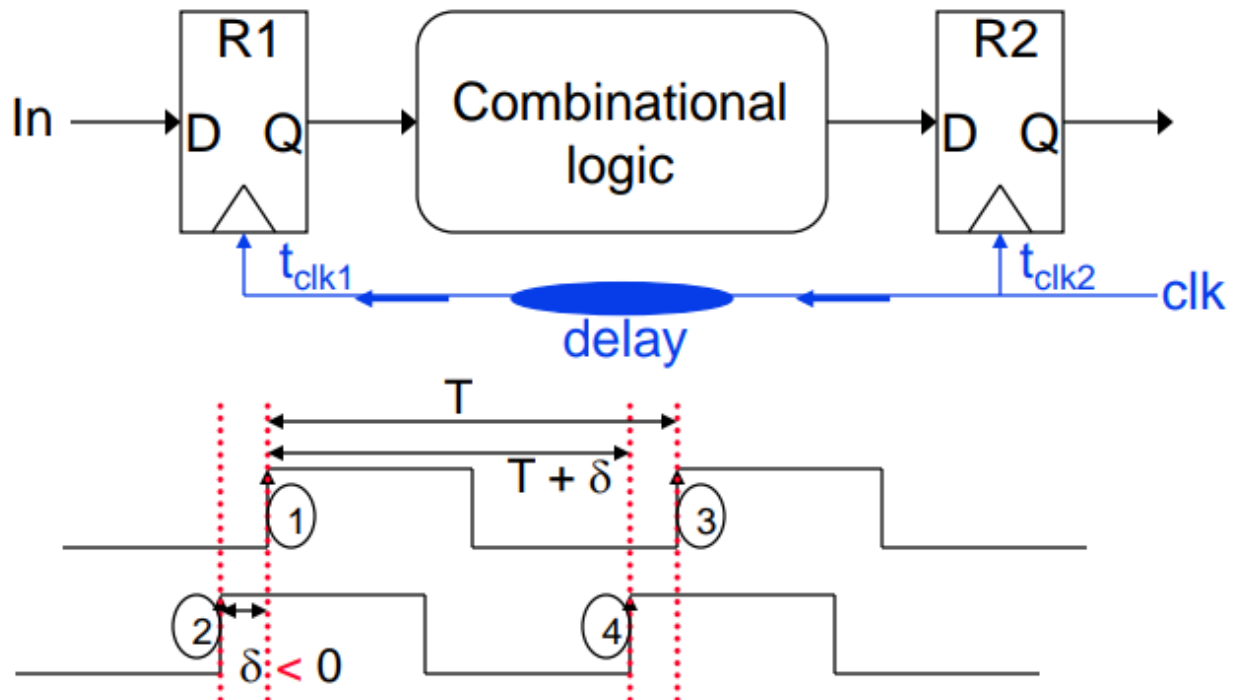
$$t_{hold}: \quad t_{hold} + \delta \leq t_{cdlogic} + t_{cdreg} \quad \text{so} \quad t_{hold} \leq t_{cdlogic} + t_{cdreg} - \delta$$

Figure 87- Effetti positivi del clock skew.

Come si può vedere le variazioni clock arrivano sempre un poco dopo a R2. Ciò ci permette di avere un periodo di clock più basso e quindi una frequenza più alta.

Ma questo fenomeno rende la seconda regola più difficile da rispettare.

Negative clock skew



$$T: \quad T + \delta \geq t_{c-q} + t_{plogic} + t_{su} \quad \text{so} \quad T \geq t_{c-q} + t_{plogic} + t_{su} - \delta$$

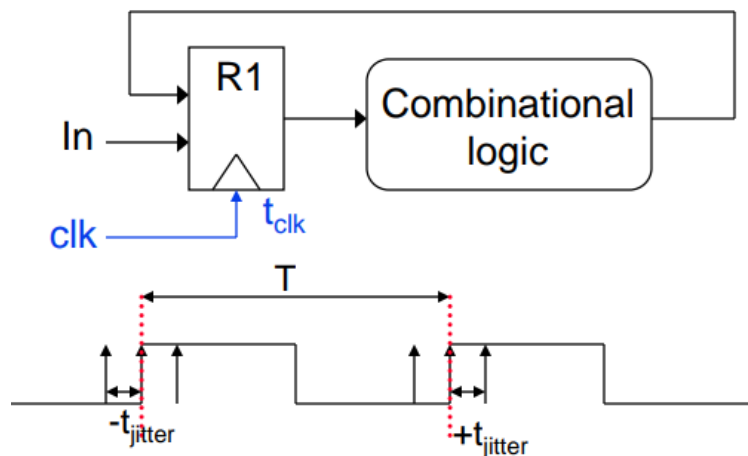
$$t_{\text{hold}} : \quad t_{\text{hold}} + \delta \leq t_{\text{cdlogic}} + t_{\text{cdreg}} \quad \text{so} \quad t_{\text{hold}} \leq t_{\text{cdlogic}} + t_{\text{cdreg}} - \delta$$

Come si può vedere le variazioni clock arrivano sempre un poco dopo a R1. Ciò ci obbliga di avere un periodo di clock più alto e quindi una frequenza più bassa.

Ma questo fenomeno rende la seconda regola più facile da rispettare.

Clock Jitter

Il jitter è dovuto al modo in cui generiamo internamente il clock, ossia attraverso il PLL (*anello di blocco di fase*). Il modo in cui lo generiamo non è perfetto e ad ogni ciclo di clock potrei avere una piccola variazione che (*fron­te di salita un po­co prima di quan­do do­vrebbe es­se­re e fron­te di dis­cesa un po­co do­po*) che ren­de la du­rata ef­fet­ti­va del pe­rio­do di clock leg­ger­men­te mag­gio­re. Ciò gio­ca a no­stro sfa­vo­re ov­via­men­te.



$$T - 2t_{\text{jitter}} \geq t_{\text{c-q}} + t_{\text{plogic}} + t_{\text{su}} \quad \text{so} \quad T \geq t_{\text{c-q}} + t_{\text{plogic}} + t_{\text{su}} + 2t_{\text{jitter}}$$

Figure 88- Effetti del jitter sulla prima regola.

Clock skew e jitter combinati

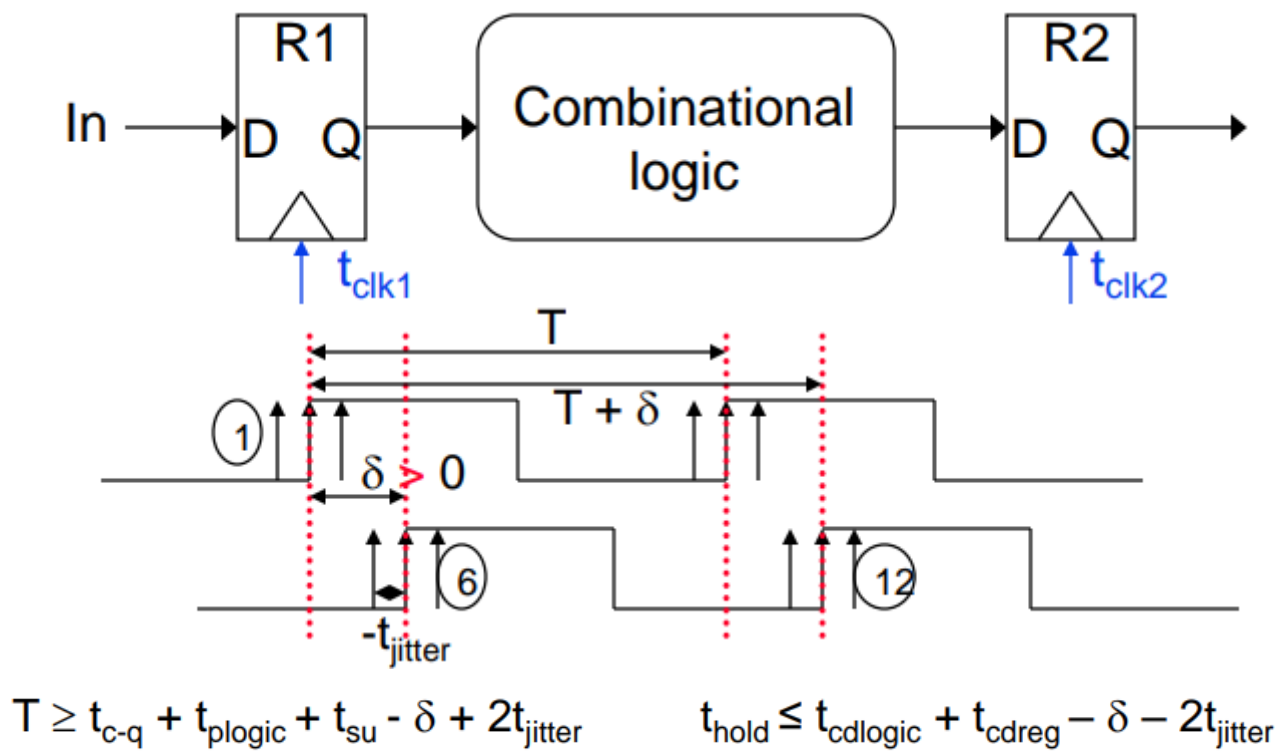


Figure 89 - effetti dello skew e del jitter insieme.

Il clock skew può rendere più facile da rispettare solo una delle due regole, mentre il jitter è un fenomeno puramente negativo che abbassa le prestazioni.

Generatore di Clock

Nota sulla Zibo board

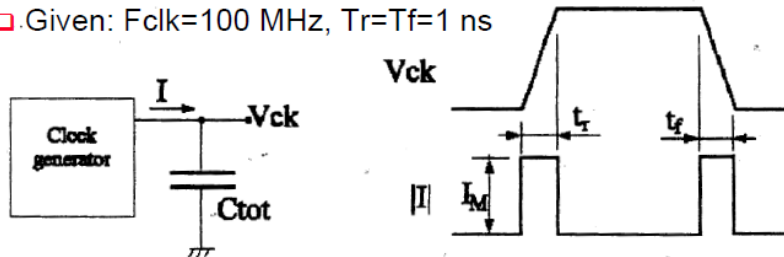
La scheda Zibo ha un FPGA e un clock di ingresso (*non ad alta frequenza, tipicamente 125 MHz*).

Come funziona il PLL? È un loop dove abbiamo il segnale di ingresso che è di 125 MHz, molto stabile, e poi se vogliamo ottenere ad esempio un clock di 400 MHz partendo da 100 MHz, usiamo un VCO (*oscillatore controllato in tensione*), dividiamo 400 MHz per 4, otteniamo un clock diviso. Possiamo avere due situazioni per il clock: se il clock è in ritardo dobbiamo aumentare la tensione (accelerare), se il clock è in anticipo dobbiamo diminuire la tensione. Questo cambiamento nella frequenza del clock generatore è chiamato jitter, che è un parametro statistico. Il clock q è stabile. Lo strumento si occupa di questo, non devi pensarci mentre progetti l'hardware.

PPL (Anello di Blocco di Fase)

Un PLL è un circuito elettronico che genera un segnale di uscita con una frequenza e una fase controllate rispetto a un segnale di riferimento di ingresso. È comunemente utilizzato per generare segnali di clock stabili e precisi per vari scopi nelle applicazioni digitali e analogiche.

- ❑ Chip with 10.000 FF
- ❑ Each FF has a $C_{in}=50$ fF for the clock input
- ❑ Clock Generator needs to drive $C_{tot}= 500$ pF
- ❑ Given: $F_{clk}=100$ MHz, $T_r=T_f=1$ ns



$$I_M = C_{tot} \cdot \left. \frac{\partial V_{ck}}{\partial t} \right|_{MAX} = C \frac{V_{dd}}{t_r} \cong 2.5 A$$

Distribuire il clock su una rete

Balanced paths

Questa tecnica mira a garantire che tutti i percorsi del segnale di clock abbiano lunghezze simili, riducendo al minimo i ritardi e le discrepanze temporali tra i vari componenti del sistema.

1. I percorsi che il segnale di clock deve percorrere vengono suddivisi in segmenti più brevi e bilanciati in modo che la lunghezza di ciascun segmento sia il più possibile simile.
 - a. Questa cosa può essere ottenuta tramite la disposizione strategica dei componenti e dei collegamenti sulla scheda o nel circuito.
2. Stimare i ritardi associati a ciascun segmento.
 - a. Si tiene conto delle caratteristiche del materiale, delle lunghezze e della velocità di propagazione dei segnali.
3. Se necessario, vengono implementate tecniche di compensazione dei ritardi per garantire che tutti i segmenti del percorso del clock siano allineati temporalmente.
 - a. Come l'aggiunta di linee di ritardo o di buffer le in punti strategici lungo il segmento.
4. Verifica tramite simulazione del perfetto bilanciamento dei percorsi.

Clock grids

Dividiamo il nostro chip in diverse aree locali, ognuna delle quali ha il suo clock locale.

Ogni area è collegata con altre aree attraverso una comunicazione asincrona.

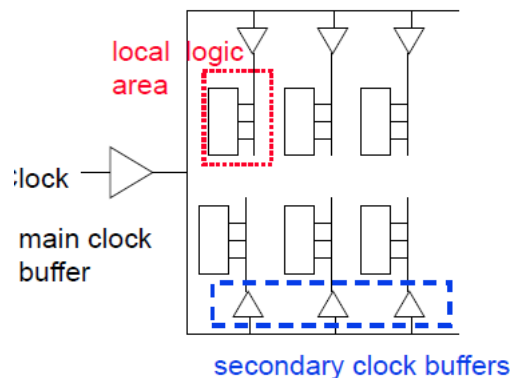


Figure 90 - Esempio di clock grids

Questo approccio serve a ridurre lo sfasamento del clock. Il nuovo problema è quello di gestire la comunicazione tra le aree, che funziona utilizzando diverse frequenze di clock.

H-Tree Clock Network

Strategia utilizzata per distribuire in modo efficiente il segnale di clock in un sistema digitale, specialmente in sistemi integrati complessi come processori, FPGA o ASIC.

Questa tecnica prende il nome dalla sua struttura a forma di "H", che è simile alla forma di un albero invertito.

Come viene progettata la rete

La rete del clock viene progettata come una struttura ad albero forma di "H", con un segmento verticale centrale (tronco) e diversi rami orizzontali che si estendono da entrambi i lati.

Questa configurazione permette di distribuire il segnale di clock in modo uniforme e simmetrico a tutti i componenti del sistema.

Segmento verticale

Il segmento verticale dell'H-Tree rappresenta il percorso principale del segnale di clock. È generalmente progettato con un'ampiezza maggiore rispetto ai rami laterali per ridurre i ritardi e migliorare l'integrità del segnale.

Rami orizzontali

I rami orizzontali dell'H-Tree si estendono da entrambi i lati del tronco centrale e si suddividono ulteriormente in segmenti più piccoli man mano che si ramificano verso l'esterno. Questi rami distribuiscono il segnale di clock a livelli più bassi di gerarchia, raggiungendo eventualmente tutti i componenti del sistema.

Segmentazione e ripetizione

Lungo il percorso dell'H-Tree, vengono utilizzati buffer di segnale o elementi di ritardo per compensare i ritardi e garantire che il segnale di clock venga distribuito uniformemente e senza perdite di integrità.

Simmetria e bilanciamento

L'H-Tree viene progettato per garantire simmetria e bilanciamento nella distribuzione del clock. Tutti i rami dell'albero hanno lunghezze simili e ritardi minimi, e se i percorsi sono perfettamente bilanciati, il clock skew diventa trascurabile.

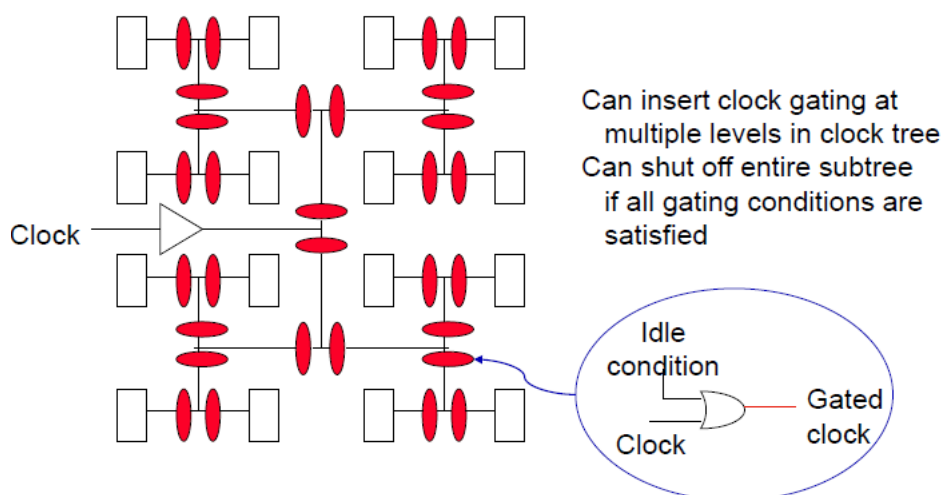


Figure 91 - Rete progettata con questa tecnica.